



Log anomaly detection in AIOps: A real-world implementation using Large Language Models

Miguel De la Cruz Cabello^a, Tiago Prince Sales^b, Marcos R. Machado^{a,*,}

^a Department of High-tech Business and Entrepreneurship (HBE), University of Twente, 7522 NB Enschede, The Netherlands

^b Department of Semantics, Cybersecurity & Services, University of Twente, 7500 AE Enschede, The Netherlands

ARTICLE INFO

Keywords:

Artificial Intelligence for IT Operations (AIOps)
Log anomaly detection
Linux systems
Large Language Models
LogBERT

ABSTRACT

This study investigates the application of Large Language Models (LLMs) for log anomaly detection within the emerging field of AIOps, where large-scale operational logs are increasingly used to support reliability engineering and automated incident response. However, deploying LLM-based anomaly detection in military environments raises practical constraints, including strict data confidentiality, limited data sharing, and frequent shifts in operational conditions and log formats. To address these challenges, we design and implement a self-supervised anomaly detection framework based on LogBERT, trained only on normal Linux syslog sequences, and deploy it locally to avoid external dependencies. We explore critical parameters, including the minimum number of tokens per log sequence, sliding window intervals, and mask ratios while attempting to detect log anomaly. In controlled experiments, a 15-second sliding window with a 10-second overlap provided the best trade-off between detection effectiveness and inference latency, supporting real-time monitoring requirements. Quantitative evaluation demonstrates that shorter sliding windows, despite capturing less context, resulted in slightly higher detection performance of anomalous logs. The model achieved high accuracy in distinguishing normal from abnormal log sequences, where sequences were classified as anomalous if more than 10% of masked tokens were incorrectly predicted. A qualitative assessment with domain experts further validated the operational usefulness of the approach, indicating reduced manual monitoring effort and suitability for integration into AIOps pipelines under confidentiality constraints.

1. Introduction

The continuous digital transformation of critical infrastructures has driven the need for intelligent systems capable of autonomously monitoring, diagnosing, and responding to complex operational events. Within this context, Artificial Intelligence for IT Operations (AIOps) has emerged as a pivotal paradigm that integrates advanced analytics and machine learning (ML) to support real-time decision-making in dynamic IT environments [1]. AIOps platforms combine data-driven learning, automation, and human expertise to detect anomalies, identify root causes, and prevent system failures across heterogeneous infrastructures. Their adoption is largely motivated by the increasing complexity and scale of modern IT ecosystems, where traditional monitoring tools and rule-based mechanisms struggle to maintain service continuity, optimize performance, and minimize downtime. Log anomaly detection is a core capability in AIOps because logs provide the most fine-grained and ubiquitous record of system behavior, including early signals of misconfigurations, performance regressions,

and security incidents. In practice, timely detection of anomalous log sequences can reduce mean time to detect and mean time to recover by enabling earlier triage, prioritization of incidents, and automated containment actions (e.g., isolating a service, escalating authentication failures, or triggering rollback procedures). This makes log anomaly detection particularly important in environments where operational continuity and rapid response are non-negotiable. Recent AI research in adjacent operational domains (e.g., smart-city service environments) similarly emphasizes robust, AI-enabled frameworks for managing heterogeneous and rapidly changing infrastructures, reinforcing the broader shift toward AI-supported operational resilience [2].

The importance of AIOps is particularly pronounced in mission-critical domains such as defense and aerospace, where operational reliability and data confidentiality are paramount [3,4]. Military systems often rely on Linux-based servers, embedded devices, and edge computing nodes to support secure communications, surveillance, and command-control applications. In these contexts, even a single anomaly

* Corresponding author.

E-mail addresses: m.delacruzcabello@student.utwente.nl (M. De la Cruz Cabello), t.princesales@utwente.nl (T.P. Sales), m.r.machado@utwente.nl (M.R. Machado).

<https://doi.org/10.1016/j.sasc.2026.200475>

Received 28 October 2025; Received in revised form 17 February 2026; Accepted 3 March 2026

Available online 5 March 2026

2772-9419/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

in a Linux syslog can propagate into cascading failures, potentially compromising mission readiness or safety. Consequently, robust and interpretable AI-driven anomaly detection solutions have become indispensable for ensuring operational resilience. In parallel, recent reviews of large language models (LLMs) for cyber resilience highlight both the opportunities (e.g., improved detection and decision support) and the constraints (e.g., robustness, trust, and deployment risk) when adopting LLM-based methods in security-critical operational settings [5].

Company X,¹ a leading organization known for innovation, digital trust, and operational excellence, has recently initiated the integration of AI technologies into its IT operations. Its strategic priorities emphasize security, transparency, and user alignment, while maintaining strict control over data pipelines and system integrity. Due to confidentiality constraints and the sensitive nature of its operations, outsourcing monitoring or anomaly detection to third-party providers is not an option. Therefore, any AIOps solution must be designed, trained, and deployed entirely in-house—ensuring full data sovereignty and adherence to rigorous security protocols.

These constraints motivate the modeling choices in this study. First, supervised approaches that require large volumes of labeled anomalies are difficult to operationalize in secure settings because labels are scarce, costly, and often delayed. Second, log formats and event templates evolve as systems are patched, reconfigured, or redeployed, which increases the risk of brittle detectors that depend on fixed rules or static feature engineering. Third, operational teams require methods that are efficient enough for near-real-time monitoring and that can be integrated into existing incident workflows without exposing sensitive data beyond organizational boundaries.

Against this background, this study investigates the design and implementation of a log anomaly detection system based on Large Language Models (LLMs), tailored to the operational requirements of a secure AIOps pipeline. Specifically, we explore how parameters such as the minimum token count per sequence, sliding-window duration and overlap, and masking ratio affect detection accuracy and inference latency in real-time environments. The study builds upon the LogBERT architecture [6], a transformer-based model leveraging self-supervised learning to detect anomalies in unstructured logs. While LogBERT has demonstrated state-of-the-art performance on structured and labeled datasets—such as Hadoop Distributed File System (HDFS), BlueGene/L (BGL), and Thunderbird—its applicability to heterogeneous and unlabeled Linux syslogs remains underexplored in the literature. We propose this model choice because LogBERT aligns with the constraints above: it learns normal log language patterns via self-supervision (reducing dependence on anomaly labels), supports sequence modeling needed to capture temporal context in syslog streams, and can be deployed locally with controlled preprocessing and inference. In addition, the masking-and-reconstruction objective provides a principled anomaly score based on unexpected token predictions, which is suitable when explicit anomaly categories are unavailable.

This research therefore extends the scientific understanding of log anomaly detection by adapting and evaluating LogBERT on both open-source and proprietary Linux-based datasets. The experimental setup required several methodological innovations, including adjustments in preprocessing, tokenization, and labeling strategies to accommodate the structural diversity of Linux syslogs. In operational terms, we define an anomaly as any log event that deviates from the system's normal behavior, ranging from low-level integrity failures (e.g., kernel panics, I/O errors, or memory corruption) to mid-level service degradations (e.g., systemd unit failures or authentication anomalies). This definition aligns with real-world operational requirements, emphasizing practical

detectability and interpretability. The importance of the proposed approach, relative to common alternatives, lies in its ability to operate under limited labels and heterogeneous log formats while remaining feasible for secure, in-house deployment. Compared with rule-based detectors and template-matching pipelines, the approach reduces manual maintenance when log messages drift. Compared with conventional supervised classifiers, it does not assume balanced labeled datasets and can be trained on normal-operation logs. Compared with generic large language model services, it avoids external data transfer by running locally and can be integrated as a component within an AIOps pipeline that enforces confidentiality and audit constraints.

Through extensive experimentation, the model achieved an F1-score of 94.25%, precision of 95.49%, and recall of 93.04% with a 15-second sliding window and a 10-second overlap—representing the optimal balance between detection performance and inference latency. This configuration also yielded the lowest median inference latency (5.13 ms), confirming its suitability for real-time monitoring scenarios. A qualitative assessment involving six domain experts from Company X's system maintenance and cybersecurity departments validated the operational relevance of the results. Experts confirmed that the model effectively captured abnormal log patterns and demonstrated clear potential for integration into broader IT monitoring and cybersecurity pipelines.

The contributions and novelty of this study are threefold. First, it introduces the first empirical evaluation of LogBERT applied to heterogeneous Linux syslogs under strict confidentiality and security constraints, addressing a significant research gap in AIOps literature. Second, it provides a methodological framework that integrates both performance optimization and latency analysis—an essential yet often overlooked aspect of real-time AIOps deployment. Third, the study incorporates qualitative validation from domain experts, bridging the gap between algorithmic performance and operational applicability. Collectively, these contributions advance the understanding of LLM-based anomaly detection in constrained and mission-critical environments, providing both scientific insights and practical implications for future AIOps implementations.

The remainder of this paper is structured as follows. Section 2 reviews the related literature on AIOps and log anomaly detection. Sections 3 and 4 describes the methodology, including the data preprocessing pipeline, experimental setup, and model configurations. Section 5 presents the empirical results, followed by an in-depth discussion in Section 6. Finally, Section 7 concludes the paper and outlines directions for future research.

2. Literature review

This literature review provides an in-depth examination of recent progress in automated IT operations (AIOps), the use of large language models (LLMs) for detecting log anomalies, and the integration of Retrieval-Augmented Generation (RAG) to enhance contextual understanding in log-based AI systems. The first subsection covers the foundational principles, evolving techniques, and practical implications of AIOps. The second explores how LLMs are being adapted for log anomaly detection, overcoming several limitations of traditional and deep learning models. The third investigates the role of RAG in augmenting LLM capabilities by incorporating external knowledge to enable more context-aware and explainable outcomes. The section concludes with a discussion of key research gaps, emphasizing the scarcity of anomaly detection solutions tailored for high-stakes domains such as defense.

2.1. AIOps: Techniques, challenges, benefits, and future trends

AIOps, or Artificial Intelligence for IT Operations, represents a modern paradigm that blends data science methods with IT service

¹ For anonymization purposes, the name of the collaborating organization has been replaced with “Company X”. This anonymization safeguards operational confidentiality and data sovereignty.

management to improve system observability and resilience. The approach employs machine learning, big data analytics, and visualization strategies to deliver actionable insights that go beyond what conventional monitoring tools can offer [1]. Its predictive capabilities allow organizations to transition from reactive maintenance to proactive intervention, resulting in cost savings and enhanced uptime [7].

The AIOps lifecycle incorporates multiple data sources — such as log entries, metrics, system traces, alert signals, and incident documentation — into a structured workflow encompassing preprocessing, anomaly detection, diagnosis, forecasting, and automated recovery [8]. These stages are interdependent, where failure detection builds upon preprocessed data and sets the stage for subsequent root cause analysis and remediation actions.

This review emphasizes anomaly detection as a critical entry point within the AIOps framework. The data preparation phase includes steps like parsing semi-structured logs, handling incomplete metrics through imputation, and abstracting inputs to emphasize relevant patterns [8]. Once transformed, the dataset can support failure prediction tasks that identify irregular system behavior using historical baselines.

Among current techniques, signal reconstruction methods such as Spectral Residuals (SR), Fast Fourier Transform (FFT), and Prediction Confidence Intervals (PCI) have become popular due to their efficiency [9]. These models reduce dimensionality by encoding time series into latent representations, although this abstraction often leads to a loss of finer-grained information. To preserve data fidelity, advanced deep learning techniques — including autoencoders, convolutional neural networks, and recurrent models like LSTMs — have demonstrated superior accuracy in identifying system anomalies [9,10].

Yet, practical implementation of these models is complicated by several persistent challenges. For instance, the evolving nature of IT environments, described as concept drift, can degrade model performance if continuous retraining is not applied [9]. Moreover, sourcing high-quality labeled data is difficult, and organizational inertia may hinder the transition to AI-based operations [7,8]. Integration into legacy systems also remains non-trivial due to variation in software architectures.

Despite these hurdles, AIOps adoption yields substantial returns: from improved reliability and user satisfaction to better productivity among development teams and reduced mean time to resolution [3,7]. It also encourages cross-functional cooperation and provides scalable solutions for managing large, heterogeneous infrastructures [1,10]. In addition, innovations such as meta-learning — particularly via model-agnostic meta-learning (MAML) — are gaining traction for their ability to generalize across unseen anomaly types by leveraging knowledge from prior tasks [11].

AIOps also aligns with broader trends in agile IT governance through synergies with DevOps, GitOps, and MLOps frameworks [3, 12]. The introduction of LLMs into AIOps marks a transformative leap, particularly for analyzing natural language-based records such as logs and incident reports. These models bring inherent capabilities in pattern recognition and language understanding, enabling them to automate diagnostic tasks and reduce reliance on manual feature engineering [8,13]. However, their deployment must consider computational scalability and domain-specific tuning requirements to fully realize their potential.

2.2. LLMs in log anomaly detection

Automated detection of anomalies in system log data has become a cornerstone for ensuring the security and reliability of digital infrastructures. Typically cast as a binary classification task, log anomaly detection distinguishes between normal and aberrant patterns in the voluminous, semi-structured outputs generated by software systems [14]. These logs are essential for performance monitoring and debugging but pose substantial challenges due to their unstructured nature, variability, and high data volume [15,16].

Compounding the issue are problems like class imbalance — since anomalies are rare by definition — and the limited availability of annotated training samples, which restricts the applicability of conventional supervised machine learning techniques [17,18]. While heuristic parsing methods such as regular expressions have traditionally been employed to segment log entries into templates, they are brittle, error-prone, and require frequent updates as systems evolve [16]. Furthermore, with many organizations maintaining heterogeneous logging formats, developing unified parsers remains a persistent obstacle.

Legacy anomaly detection methods have generally relied on statistical models or tree-based classifiers like PCA, Isolation Forests, and SVMs [9,15]. These approaches require manual feature selection and struggle with generalization across domains. Deep learning approaches such as LSTMs and transformer-based architectures have partially addressed this by learning temporal and contextual dependencies from sequential logs. Notably, LogBERT [6] employs a self-supervised masked language modeling objective combined with hypersphere minimization to detect deviations from expected patterns. It uses the Drain algorithm for parsing logs in real time, showing superior performance across benchmarks like HDFS and Thunderbird [19].

Recent innovations pivot toward LLMs to enhance anomaly detection by leveraging their semantic interpretability. Systems like LogLLM [18] replace static parsing with dynamic object masking and vector projection techniques, achieving higher F1-scores compared to traditional baselines. LogFormer [14] and OWL [20] emphasize transferability across domains using attention mechanisms and tuning strategies such as Low-Rank Adaptation (LoRA). Meanwhile, Xu and Ding [21] propose a taxonomy distinguishing prompt-based and contrast-based anomaly detection methods, incorporating tools such as in-context learning and chain-of-thought prompting.

Alternative strategies, such as LogFIT [22], eliminate dependency on parsing altogether by applying masked token prediction to raw logs. This approach reduces parsing errors and improves adaptability to out-of-vocabulary terms. Similarly, LogPrompt [23] explores various prompt engineering tactics to enhance transparency and interactivity in anomaly detection workflows.

Incorporating external knowledge has also shown promise. Tools like LogExpert [24] integrate domain-specific insights from developer forums, while RAPID [25] offers a training-free, retrieval-based anomaly detector capable of identifying semantic deviations using token-level comparisons. While these methods demonstrate impressive results on public datasets, they also raise new concerns related to privacy, cost, and system integration [16].

2.3. Retrieval-Augmented Generation (RAG)

Pretrained LLMs are widely known for their expansive general knowledge, but their static nature and lack of domain specialization can result in factual inaccuracies or hallucinations. To address these limitations, Retrieval-Augmented Generation (RAG) introduces an external knowledge retrieval step that supplements model outputs with contextual information [26,27]. This approach avoids full model re-training by injecting relevant, up-to-date content into prompts, making it particularly valuable for dynamic environments like IT operations.

RAG systems typically follow a three-phase process: indexing, retrieval, and generation. First, raw data is preprocessed and transformed into a uniform format suitable for embedding. During retrieval, semantically similar documents are identified using techniques such as cosine similarity. Finally, the most relevant documents are embedded in the LLM prompt, enabling context-aware generation [27]. Dense retrieval methods, such as Dense Passage Retrieval (DPR), offer higher semantic fidelity compared to sparse methods like BM25, though both have use cases [8].

Evaluation frameworks for RAG include retrieval metrics like Mean Reciprocal Rank (MRR), Hit Rate (HR), and Normalized Discounted Cumulative Gain (NDCG), alongside output quality metrics such as

BLEU, ROUGE, and F1 [27]. Recently, automated evaluation platforms like RAGAS and ARES have been proposed to benchmark end-to-end performance using LLM scoring.

RAG applications span a variety of domains. In agriculture, Vizniuk et al. [28] use RAG to aid decision-making in crop management and pest control, overcoming data heterogeneity and knowledge scarcity. In manufacturing, Jeon et al. [29] introduce ChatCNC, which connects operational databases with LLMs to enable real-time interaction through SQL. For IT operations, Zhang et al. [4] present RAG4ITOps, a two-stage system combining supervised embedding learning with contrastive techniques and vector retrieval using FAISS. Similarly, Chen et al. [30] deploy OMAI, a Q&A system specialized in high-energy physics, using fine-tuned embeddings and helpdesk data to enhance operational support.

Together, these implementations demonstrate RAG's capacity to enrich LLMs with task-relevant knowledge, improve interpretability, and adapt to specialized operational contexts. As an architectural framework, RAG not only supplements fine-tuning but may serve as a foundational design pattern for next-generation context-aware AI systems.

2.4. Gaps in the literature

Despite rapid advancements in AIOps, LLM-based log analysis, and RAG frameworks, there remains a notable gap in the application of these technologies to highly sensitive or mission-critical domains such as military and defense operations. Most state-of-the-art techniques are benchmarked on publicly available datasets (e.g., HDFS, BGL, Thunderbird), which are limited in complexity and do not reflect the operational constraints, adversarial conditions, or heterogeneity of logs encountered in military environments.

In particular, data-driven anomaly detection approaches that incorporate domain-specific semantics, handle multi-source and encrypted log formats, or operate under resource-constrained and disconnected settings remain underexplored. The lack of open-source military-grade datasets exacerbates this issue, hindering both reproducibility and methodological innovation. Consequently, the development of robust, interpretable, and privacy-preserving models tailored for security-sensitive operations represents an important frontier for future research.

3. Methodology

This section provides a comprehensive analysis of the methodology employed in the log anomaly detection study within the context of AIOps. The methodology serves as a guiding framework that shapes the research process. A structured and rigorous methodology is fundamental to ensuring the effectiveness, transparency, and reproducibility of research objectives. This section is organized into three main sections: DSR, CRISP-ML, and analytical methods. The DSR section describes the practical research strategy, highlighting how the system is developed and evaluated, while the CRISP-ML section details the methodological framework guiding the research process. Then, the analytical methods section describes the technical aspects, specifically outlining the baselines, metrics, and models used for implementing the log anomaly detection system. By dividing the methodology section into these separate subsections, the study aims to provide a complete overview of the whole research framework.

3.1. Methodological framework

This study adopts a hybrid methodological framework that combines Design Science Research (DSR) with the CRISP-ML(Q) process model to guide the systematic design, development, and evaluation of a log anomaly detection system in mission-critical IT environments.

DSR provides the overarching structure for creating and evaluating artifacts that address real-world problems in Information Systems [31]. It is particularly suitable for this context, as the goal is to

design a real-time, scalable, and adaptive system capable of detecting anomalies in unstructured logs. Following the six activities defined by Vom Brocke et al. [31], we (i) identified limitations in traditional rule-based anomaly detection, (ii) defined solution requirements such as low-latency and privacy-preserving inference, (iii) developed an AI pipeline capable of learning from log patterns, (iv) demonstrated functionality using benchmark datasets, (v) evaluated performance through accuracy metrics and expert feedback, and (vi) communicated results to stakeholders through documentation and presentations.

Complementing DSR, the CRISP-ML(Q) methodology [32] was applied to ensure a quality-assured ML lifecycle. Originally built on CRISP-DM, CRISP-ML(Q) addresses modern ML challenges like concept drift, continuous monitoring, and real-time deployment. The six-phase cycle — business understanding, data preparation, modeling, evaluation, deployment, and monitoring — was adapted to our context. Key activities included the transformation of raw logs into structured formats, training a lightweight LLM on historical data, evaluating against established baselines, and outlining a deployment-ready MVP that meets confidentiality and scalability needs.

Together, these methodologies provided a rigorous yet practical foundation for designing a robust and generalizable anomaly detection solution that aligns with both academic research standards and real-world operational requirements.

3.2. Analytical methods

This section outlines the analytical techniques applied for log parsing and anomaly detection. Each method is briefly described to provide an overview of its purpose, functioning, and relevance to the project. References to original works are retained for readers seeking more technical depth.

3.2.1. Drain

Drain is an online log parsing algorithm that transforms raw, unstructured log entries into structured templates without requiring batch processing [19]. It uses a fixed-depth parse tree structure that organizes logs by message length, initial constant tokens, and token similarity, enabling fast and scalable grouping of log entries in real time.

In our study, Drain served as the preprocessing layer for log data. Its ability to structure logs incrementally with low overhead made it suitable for streaming environments and aligned well with the needs of anomaly detection models that rely on consistent input formats.

3.2.2. Isolation forest

Isolation Forest is an unsupervised anomaly detection algorithm that isolates outliers by recursively partitioning data using randomly selected attributes and split values [33]. Because anomalies tend to differ from the majority of data, they are isolated quickly and exhibit shorter path lengths in the resulting trees.

We used Isolation Forest as a baseline method for anomaly detection due to its simplicity and robustness to high-dimensional data. It does not rely on distance or density assumptions, making it applicable to varied log data distributions in our experiments.

3.2.3. Principal Component Analysis (PCA)

PCA is a linear dimensionality reduction technique that identifies the principal axes of variation in a dataset and projects data onto a lower-dimensional space [34]. It aims to preserve as much variance as possible while minimizing reconstruction error from the reduced representation.

In anomaly detection, PCA is used to reconstruct input data; large deviations from this reconstruction are flagged as anomalies. Its low complexity and interpretability made it a useful unsupervised baseline in our evaluation.

3.2.4. Decision tree

Decision Trees are supervised learning models that solve classification or regression tasks by recursively partitioning data based on feature values [34]. Each path from root to leaf represents a decision rule, and terminal nodes correspond to predicted outcomes.

While interpretable and fast to train, Decision Trees are prone to overfitting and less robust to noise in high-dimensional log data. For our purposes, they served as a baseline for comparison with more advanced sequential models.

3.2.5. LSTMs

LSTMs are a specialized type of recurrent neural network designed to model long-range dependencies in sequential data [35]. They address the vanishing gradient problem using memory cells and gates (input, forget, and output) that manage the flow of information over time.

In the context of log anomaly detection, LSTMs are well-suited to capture temporal correlations in log sequences. We considered LSTMs as part of the comparative analysis but opted for transformer-based alternatives due to their parallelization advantages.

3.2.6. Transformer-based architectures

Transformers, introduced by Vaswani et al. [36], rely on a self-attention mechanism to model dependencies between all tokens in a sequence simultaneously, enabling efficient parallel processing and better modeling of long-range relationships.

These architectures have become foundational for modern NLP and log analytics. Their scalability and contextual awareness make them ideal for modeling unstructured log data, particularly when combined with pretraining strategies.

3.2.7. BERT

BERT [37] is a bidirectional transformer model pre-trained using masked language modeling (MLM) and next sentence prediction (NSP) tasks. It learns contextual embeddings from large corpora, enabling high performance across a range of NLP tasks.

Its modular design allows for task-specific fine-tuning without changing the underlying architecture. BERT inspired domain-adapted models like LogBERT, which leverage its strengths for structured log sequences.

3.2.8. LogBERT

LogBERT [6] adapts BERT for unsupervised log anomaly detection by using masked log prediction and hypersphere minimization. It learns semantic relationships among log keys and identifies anomalies based on prediction errors. Drain is used to preprocess logs before encoding.

We selected LogBERT for this project due to its efficiency, unsupervised training, and alignment with real-time monitoring requirements. Its performance on datasets like HDFS, BGL, and Thunderbird surpasses classical models such as PCA, DeepLog, and LogAnomaly [6], making it well-suited for operational deployment under resource constraints.

3.3. Evaluation metrics

To assess the performance of the anomaly detection models, we employed standard classification metrics derived from the confusion matrix, a tabular summary comparing predicted and actual class labels [34]. The confusion matrix consists of four components: true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN). These values offer a comprehensive view of model performance across both correct and incorrect predictions.

Based on the confusion matrix, we calculated three key evaluation metrics: precision, recall, and F1-score. Precision quantifies the proportion of correctly identified anomalies among all instances labeled as anomalous, effectively measuring the model's false positive rate. Recall, on the other hand, indicates how many of the actual anomalies were correctly detected, capturing the false negative rate. The F1-score

serves as a harmonic mean between precision and recall, offering a balanced measure particularly valuable in imbalanced datasets, where anomaly cases are rare relative to normal cases.

In addition to classification-based evaluation, we also employed cosine similarity during the anomaly labeling phase. This metric measured the semantic closeness between log messages predicted by the model and a curated list of known anomaly indicators. By comparing the vector representations of two log entries in the embedding space, cosine similarity offers insight into how well the model captures the contextual meaning of an anomaly, beyond binary classification.

Using both statistical and semantic evaluation allowed for a more nuanced assessment of model performance. While precision, recall, and F1-score ensured the reliability of detection from a traditional classification standpoint, cosine similarity complemented them by validating the alignment of log semantics in the context of masked language modeling. Together, these metrics provided a comprehensive and interpretable framework for evaluating real-time log anomaly detection systems.

4. Experimental set up

To clarify the steps of the proposed approach, the experimental pipeline follows six main stages: (1) data collection from open-source and proprietary Linux syslog sources; (2) log parsing and normalization into structured templates and EventIds; (3) anomaly labeling used only for dataset splitting and evaluation; (4) time-based sliding-window construction and vocabulary building to create model-ready sequences; (5) self-supervised training of LogBERT using only normal windows via masked-token prediction; and (6) anomaly detection and evaluation, where windows are flagged based on mask-prediction errors and assessed using standard classification metrics and expert review. These stages correspond to the end-to-end process from raw log ingestion to validated anomaly outputs. For transparency and reproducibility, each stage is also documented with a dedicated flowchart that shows inputs, processing operations, and outputs; these diagrams provide the detailed step-by-step procedures referenced throughout this section.

This section outlines the experimental setup, describing the approaches used throughout the project for log anomaly detection. The main objective is to establish a reproducible and transparent process that demonstrates how raw log messages are transformed, modeled, and evaluated to validate the effectiveness of our approaches under real-world conditions. The section begins with a description of the datasets collected, including their original source, structure, and relevance. The data processing steps are then detailed, discussing how raw logs were cleaned, parsed, and encoded into a suitable format for model training. Finally, the section explains the experiments with the training and testing procedures, including the evaluation metrics, baseline comparisons, and hyperparameter configurations used to achieve the model's effectiveness.

To support clarity and reproducibility, we include custom process flowcharts for each stage of the pipeline. Although visually inspired by standard business process diagrams, these flowcharts do not adhere to any formal BPMN² standard. Instead, we use a color-coded legend where green represents the input data, yellow represents the processing steps, and purple indicates the output. The flowcharts distinguish between types of operations by using different shapes: rounded rectangles are data processing steps, parallelograms indicate not stored data, and the cylinder symbol represents stored data.

4.1. Data overview

This section gives an overview of the datasets used for this project, including their sources, structure, and distribution. The data overview

² BPMN (Business Process Model and Notation) is a standardized graphical notation for modeling business processes.

Table 1
Syslog messages overview

Metric	Company X	Open-source	% (Company X)	% (Open-source)
Total logs	48 077	2000	100	100
Normal logs	47 362	1953	98.51	97.65
Anomalous logs	715	47	1.49	2.35

aligns directly with the *Problem Identification and Motivation* and *Define Objectives of a Solution* steps of the DSR methodology in Section 3.1 and the *Business and Data Understanding* step of the CRISP-ML methodology in Section 3.1.

4.1.1. Data collection and description

The process of collecting the Linux datasets involved two main sources. First, a publicly available dataset was acquired from Loghub [38], a large collection of system log datasets. It was originally curated by Dr. Anton Chuvakin and published as part of the *Public Security Log Sharing Site* project. This dataset contains 2000 syslog messages from the `var/log/messages` txt file of a RedHat/Fedora Linux server, collected over a span of more than 260 days. The logs indicate that this server was running a range of services, including SSH, FTP, SNMP, CUPS, Cyrus mail services, and system utilities, such as `logrotate`. There are also external IP addresses, suggesting that the server was not in an isolated environment but instead was directly exposed to the Internet. Furthermore, the ownership of the server is not documented in the dataset. Secondly, a proprietary dataset was collected through collaboration with Company X's internal teams responsible of managing and monitoring log files. Through this collaboration, we gained access to a sample of syslog sequences coming from the `var/log/messages` txt file of an operational control machine used in critical operations. The dataset consisted of 48,077 log messages, covering approximately one month of system activity, from February 21, 2025, to March 19, 2025. By using this real-world dataset, this study aims to evaluate the model's ability to generalize beyond curated environments, thereby validating its effectiveness in operational settings.

These Linux datasets consisted of system-generated messages that capture kernel events, service activity, error reports, failures, and other operational behaviors. Typically stored in files such as `var/log/messages`, these logs provide a chronological and semi-structured view of system operations. Their detailed nature and temporal ordering make them a critical resource for failure management processes, including anomaly detection.

Stakeholders such as software and system architects, developer engineers, DevOps engineers, system engineers, AI scientists, team leaders, and project managers highlighted how anomalies in Linux-based systems could compromise mission-critical operations, recognized the need for an AI system capable of real-time log analysis, and clarified operational requirements and latency constraints.

4.1.2. Data distribution

To better understand the nature of the data sets, we performed an analysis of their message distributions. Table 1 summarizes this distribution for both Linux datasets. There are 48,077 annotated syslog messages from the Company X dataset and 2000 syslog messages from the Dr. Anton Chuvakin [39] dataset. Among the syslog messages from Company X, there are 47,362 normal logs and 715 abnormal instances. In comparison, the dataset from Dr. Anton Chuvakin includes 1953 normal logs and 47 abnormal instances.

To further analyze the structure of the datasets, we performed an additional analysis (see Table 2) on the log sequences after applying the time-based sliding window. The Company X dataset generated 5339 windows, of which 5066 were labeled normal and 273 anomalous. In contrast, the open-source dataset generated 230 windows, with 186 labeled normal and 44 as anomalous.

Table 2
Overview of windows.

Metric	Company X	Open-source	% (Company X)	% (Open-source)
Total windows	5 339	230	100	100
Normal windows	5 066	186	94.80	80.87
Anomalous windows	273	44	5.10	19.13

Table 3
Top 10 most frequent log templates from Company X dataset.

Rank	EventTemplate	Occurrences	%(Company X)
1	Template ₁ : time synchronization operations	8926	18.57
2	Template ₂ : adjustments in system clocks	6060	12.60
3	Template ₃ : successful execution of system-level tasks	5235	10.89
4	Template ₄ : startup of a system monitoring	3762	7.82
5	Template ₅ : initiation of core system services	3762	7.82
6	Template ₆ : network failures	1459	3.03
7	Template ₇ : hardware detection and classification processes	1206	2.51
8	Template ₈ : device status updates	741	1.54
9	Template ₉ : local requests within the system	660	1.37
10	Template ₁₀ : network-level disturbances	400	0.83

Table 4
Top 10 most frequent log templates from open-source dataset.

Rank	EventTemplate	Occurrences	%(Open-source)
1	connection from <*> <*> at <*> <*> <*> <*>: <*> <*> <*>	909	45.45
2	authentication failure; logname= uid=<*> euid=<*> tty=NODEVssh ruser=<*> <*>	372	18.60
3	session opened for user <*> by <*>	123	6.15
4	session closed for user <*>	123	6.15
5	authentication failure; logname= uid=<*> euid=<*> <*> ruser= <*>	118	5.90
6	check pass; user unknown	117	5.85
7	ALERT exited abnormally with <*>	43	2.15
8	Kerberos authentication failed	23	1.15
9	notify question section contains no SOA	16	0.80
10	Authentication failed from <*> (<*>): Software caused connection abort	15	0.75

Moreover, to understand typical log patterns and system behavior in both datasets, we analyzed the top 10 most frequently occurring templates and the top-most frequent anomalous templates. Table 3 shows the templates that are used the most frequently in the Company X dataset. These logs primarily represent routine system operations related to time synchronization, system management, hardware interaction, and network status. Then, Table 4 lists the most frequent log templates from the open-source Linux dataset. These logs are predominantly related to authentication and security events. Table 5 lists anomalous templates from the Company X dataset that mainly reflect failures in service activation, device communication, and internal component stability. Lastly, Table 6 shows a set of anomalous templates from the open-source dataset that represent failures or misconfigurations affecting core components of the system.

4.2. Data processing

In the next sections, we apply data processing to both Linux datasets, aligning with the *Design and Development* step of the DSR methodology in Section 3.1 and the *Data Preparation* set of the CRISP-ML methodology in Section 3.1. These sections reflect the phase in

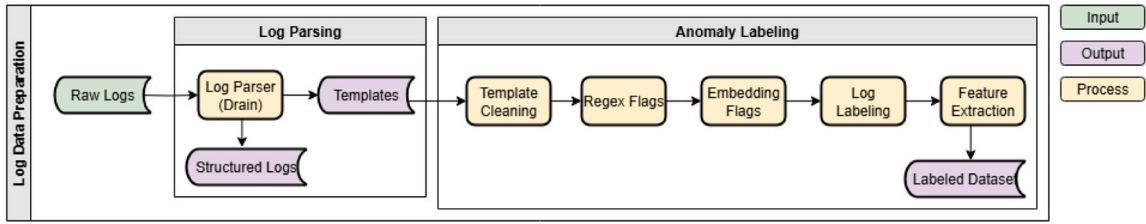


Fig. 1. Log data preparation flowchart.

Table 5

Top 10 most anomalous log templates from Company X dataset.

Rank	EventTemplate	Occurrences	%(Company X)
1	Template ₁ : hardware or filesystem-level error	131	0.27
2	Template ₂ : misconfigured SELinux policy	104	0.22
3	Template ₃ : control flow interrupted internally	47	0.1
4	Template ₄ : unrecognized service or abnormal process	44	0.09
5	Template ₅ : failure that could compromise task flows	27	0.06
6	Template ₆ : service-level unresponsiveness	22	0.045
7	Template ₇ : unidentified system. Service malfunction	21	0.04
8	Template ₈ : integrity of I/O communication or device drivers	21	0.04
9	Template ₉ : system stability issue	20	0.04
10	Template ₁₀ : coordination breakdown in service management	16	0.03

Table 6

Top most anomalous log templates from open-source dataset.

Rank	EventTemplate	Occurrences	%(Open-source)
1	ALERT exited abnormally with <*>	43	2.15
2	SELinux: Starting in permissive mode	1	0.05
3	Failure registering capabilities with the kernel	1	0.05
4	There is already a security framework initialized, register_security failed	1	0.05
5	PCI: Invalid ACPI_PCI IRQ routing table	1	0.05

which the raw data is transformed to support the development of the system, and serve as a foundation for the implementation and evaluation phases of the system.

4.2.1. Log data preparation

The log data preparation process involved transforming the raw syslog messages into a structured, labeled form. At the highest level, this transformation consisted of two sequential stages: parsing raw messages into a consistent format (log templates), and identifying which log templates correspond to anomalous behavior (see Fig. 1).

4.2.2. Log parsing

The initial step used Drain [19] as the parser algorithm to convert the raw logs into a template-based structure. When the script was called, it constructed an object configured with a specific log-format string (e.g., <Month> <Date> <Time> <Level> <Component> <Content>) as well as a set of regular expressions designed to collapse noisy tokens such as IP addresses, hexadecimal identifiers, file paths, urls, temporal and regional indicators, and stand-alone numbers into

Table 7

Top 10 anomalous components by dataset.

Rank	Company X dataset		Open-source dataset	
	Component	Occurrences	Component	Occurrences
1	systemd-vconsole-setup	134	logrotate	43
2	systemd	123	kernel	4
3	kernel	120		
4	bash	72		
5	journal	51		
6	dbus-daemon	39		
7	pulseaudio	31		
8	logrotate	27		
9	NetworkManager	22		
10	rsyslogd	18		

placeholders such as <*>. We considered these tokens noisy primarily because they showed high variability across log messages that would otherwise be structurally identical, without changing the underlying meaning of the message. For example, two logs reporting the same error may differ only in the IP address or timestamp, reflecting a change in context rather than in structure or behavior. Retaining such dynamic tokens would lead to excessive fragmentation of log templates, reducing their generalizability and increasing the number of unique templates. By converting them into placeholders, the parser was able to identify the underlying structural similarity among logs and group them under a common template. We also generated a template file with the EventTemplate and the EventId.

4.2.3. Anomaly labeling

Once the logs have been parsed, we assigned anomaly labels to the structured dataset. Although LogBERT [6] itself is trained in a completely unsupervised way, using only normal log sequences, labels are nevertheless computed at this stage for splitting purposes. To ensure clarity and consistency in the labeling, we adopted a focused definition of what constitutes an anomaly from the literature and discussions with stakeholders. We refer to a log message as an anomaly when it signals a deviation from the expected internal logic of the system. This includes both low-level integrity failures, such as kernel crashes, memory corruption, I/O errors, process crashes, and mid-level service degradations, such as system unit failures, service in failed state, network timeouts, capacity exhausted, and critical watchdog timeouts. These anomalies are often logged in the components shown in Table 7.

To assign these temporary anomaly labels, we created a multi-stage filtering and embedding procedure for the templates. First, the textual representation of each template was cleaned by removing placeholder tokens, non-alphabetical punctuation, and converting all text to lowercase. Next, we tested templates using regular expressions designed to capture high-signal error keywords, such as “panic”, “error”, “seg-fault”, “oom”, “kill”, “fail”, “alert”, “fatal”, “critical”, “crash”, and others commonly associated with system failures or critical conditions. Any template matching any of these error tokens was immediately flagged as anomalous. For the remaining unlabeled templates, we made a semantic similarity pass: both these templates and a predefined list of

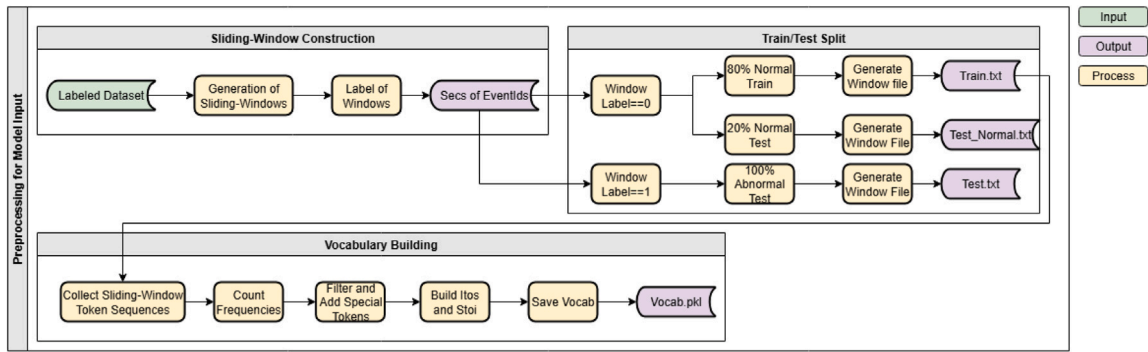


Fig. 2. Preprocessing for model input flowchart.

high-signal anomaly phrases were embedded using a lightweight LLM known as all-minilm:33m³ via the Ollama API.⁴ Cosine similarity scores were calculated between each unlabeled template and each phrase, and if the maximum similarity for a given template exceeded a threshold of 0.7, then that template was flagged as anomalous. We retrieved regular expressions and the list of anomalous phrases from Red Hat Documentation⁵ and Red Hat Customer Portal, accessible through a company's account. We also used Company X's internal GPT-based model to enrich the list whenever we encountered ambiguous logs that were difficult to label.

At the end of these steps, the raw log messages were transformed into a labeled feature dataset. Each row corresponded to a log line, augmented with an anomaly label and a timestamp. These labels were used solely to split the datasets in the next stages so that LogBERT's [6] unsupervised training could be restricted to only sequences that contained normal events. Therefore, by ensuring that the model never saw any anomalous sequence during training, we preserved the integrity of an unsupervised learning process. In our case, we only used EventIds as features. This choice had some advantages. First, it reduced dimensionality because we abstracted raw logs into unique identifiers, leading to faster training and reducing memory usage. Second, EventIds captured the structural logic of system behavior, so the model could focus on learning patterns in a sequence of events. And finally, EventIds enabled consistent input sequences required for LogBERT's masked log prediction task.

4.3. Preprocessing for model input

The next phases of the data processing pipeline consisted of changing the labeled dataset to a suitable form for model training. This involved three steps: generating time-based sliding windows to simulate a real-time log flow, splitting the datasets into training and testing, and building a vocabulary of tokens. These steps preserved the temporal structure of the data and aligned with the model's token-level input format (see Fig. 2).

4.3.1. Sliding-window construction

To simulate real-time behavior, we segmented the logs into overlapping, time-based windows. Each window spanned a fixed duration of 15 s with a 10-second overlap. To ensure that each window contained enough contextual information for masked log prediction, we temporarily stored windows with fewer than 10 tokens in a buffer

and combined that short window with the subsequent window. This buffering approach maintained a minimum sequence length while preserving temporal continuity across logs. Guo et al. [6] followed a similar approach but with larger time windows between 1 to 5 min, and instead of following our buffering technique, they just dropped windows shorter than 10 tokens. However, dropping shorter sequences meant that potential anomalies in short windows could not be evaluated by the model. Our approach ensured that even brief sequences were retained for analysis.

Classifying entire windows instead of individual logs aligns with the model's input structure and temporal dependencies that it tries to capture. Since anomalies often occur from combinations of events rather than single log lines, predicting the entire window allows the model to detect contextual deviations more effectively.

Then we assigned a binary label to each window based on its content. If all events within the window were labeled as normal, then the window was labeled as normal. If even a single anomalous event was present in the window, the whole window was labeled as abnormal. These labels were not used during training, but as a crucial step for splitting the datasets.

4.3.2. Train/test split

After labeling the windows, we divided them into training and testing sets. We allocated 80% of the normal windows for training purposes, used the remaining 20% for testing on unseen normal data, and reserved all anomalous windows exclusively for testing. We followed the same splitting approach as [6]. Lastly, each split was written into a separate txt file containing the corresponding sequences of EventIds.

4.3.3. Vocabulary building

In the next step, we transformed the raw EventId tokens into numerical inputs for the model. The vocabulary maps the tokens to indexes, and this mapping allows BERT [37] to embed each token as a dense vector. The first step was to count the frequency of each token in the training dataset. In our case, the frequency was set to 1, so every EventId was added to the vocabulary. Before sorting the tokens by frequency, LogBERT [6] reserves spaces for a set of special tokens:

- **<pad>**: Assigned to index 0, it is used to fill shorter sequences so that all the examples in the minibatch share the same length.
- **<unk>**: Assigned to index 1, it mapped any token not seen during vocabulary construction.
- **<eos>**: Assigned to index 2, it marked the end of a sequence.
- **<sos>**: Assigned to index 3, it marked the start of each input sequence before feeding it to BERT [37].
- **<mask>**: Assigned to index 4, this token was used to randomly replace real tokens and train the model to predict them.

Once we had built the list of unique tokens, we sorted them in descending order and assigned each token a unique index. Because the

³ All-minilm:33 m is a compact transformer model for efficient embeddings. More: <https://ollama.com/library/all-minilm>.

⁴ Ollama API enables local LLM access via Python. More: <https://github.com/ollama/ollama-python>.

⁵ Red Hat Documentation provides extensive technical references and troubleshooting guides for enterprise systems. More information: <https://docs.redhat.com/en>.

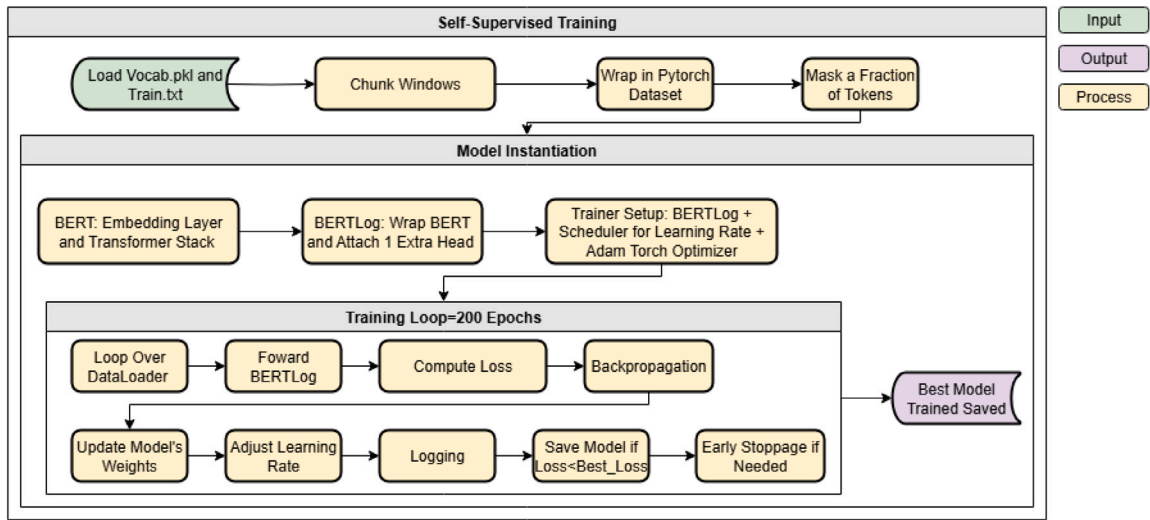


Fig. 3. Training flowchart.

embedding layer requires contiguous integer indices instead of raw integers, we created a stoi (string-to-index) dictionary that mapped each token to its index so the model could look up the correct embedding vector during training. In addition, we built an itos (index-to-string) list so that when the model outputs a predicted index, such as flagging an EventId as anomalous, we could translate that index back into the original token for interpretation. Lastly, we saved the vocabulary to a Pickle file to preserve the objects.

4.4. Training

In this section, we describe the self-supervised training process of our LogBERT model, specifically tailored for anomaly detection on Linux syslog data, and detail the model instantiation and the iterative training loop setup (see Fig. 3). The pseudocode of the training is described in Algorithm 1. This section aligns with the *Design and Development* step of the DSR methodology in Section 3.1 and the *Modeling* step of the CRISP-ML methodology in Section 3.1. The stage emphasizes the development of the model as a core component of the log anomaly detection system.

Algorithm 1 Self-Supervised Training of LogBERT

- 1: **Input:** Training and vocabulary files
- 2: **Output:** Trained model
- 3: Load vocab and training windows
- 4: Chunk windows into a fixed length size
- 5: Wrap windows in a PyTorch Dataset and DataLoader
- 6: Randomly mask tokens per window
- 7: Set training parameters:
- 8: **for** each epoch **do**
- 9: **Training phase:**
- 10: **for** each batch in training data **do**
- 11: Forward pass
- 12: Compute masked-token loss
- 13: Backpropagation and update weights
- 14: **end for**
- 15: **Validation phase:**
- 16: **for** each batch in validation data **do**
- 17: Forward pass and compute validation loss
- 18: **end for**
- 19: **end for**
- 20: **Return:** Trained model

4.4.1. Self supervised training

Initially, we loaded the vocabulary and the training data. We then segmented the log data into windows based on a maximum sequence length, which represented the number of tokens the model could process in a single window. These windows were encapsulated into a PyTorch dataset to facilitate batch processing. Furthermore, we applied the mask strategy of BERT where a percentage of tokens within each window were randomly masked, allowing the model to learn context more effectively through the MLM task.

Next, we instantiated our LogBERT model. The architecture was built upon BERT that included an embedding layer and a stack of transformer blocks. Each block included a multi-headed self-attention mechanism and a feed-forward network to capture complex dependencies in the log sequences. We described the detailed mathematical functions and structural explanations of the transformer components in Section 3.2 (Analytical Methods). Furthermore, LogBERT extended the standard BERT structure by adding an extra head designed for masked log prediction, defined mathematically as:

$$\hat{y}_{[MASK_i]}^j = \text{Softmax}(\mathbf{W}_c \mathbf{h}_{[MASK_i]}^j + \mathbf{b}_c), \quad (1)$$

where $\mathbf{h}_{[MASK_i]}^j$ is the contextual embedding vector of the masked log token, and $\mathbf{W}_c$ and $\mathbf{b}_c$ are trainable parameters. The training objective function for this task employed cross-entropy loss:

$$\mathcal{L}_{MLP} = -\frac{1}{N} \sum_{j=1}^N \sum_{i=1}^M y_{[MASK_i]}^j \log \hat{y}_{[MASK_i]}^j, \quad (2)$$

where $y_{[MASK_i]}^j$ indicates the true log, N represents the number of windows, and M the number of masked tokens per window. The training objective was that the model accurately predicted the randomly masked logs in the windows.

We set the training loop to 200 epochs. Within each epoch, we looped through batches of log windows loaded via Pytorch's DataLoader.⁶ During each interaction, our model executed a forward pass to generate predictions, after which the cross-entropy loss was computed for model optimization. We used the Adam optimizer and a learning rate scheduler that adjusted the learning rate based on the number of training steps to improve efficiency and model convergence.

⁶ The DataLoader is a PyTorch utility that efficiently loads and batches data during training and evaluation. More information: https://pytorch.org/tutorials/beginner/basics/data_tutorial.html.

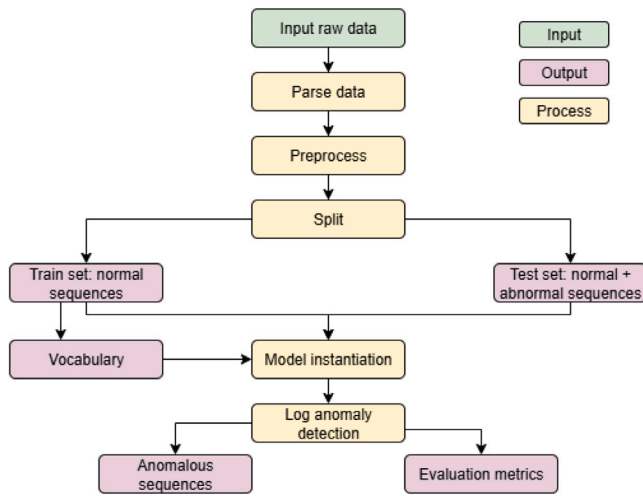


Fig. 4. Anomaly detection pipeline.

Table 8
Isolation-Forest parameters.

Parameter	Value	Description
n_estimators	200	Number of base trees in the ensemble
max_samples	Auto	Number of samples drawn to train each tree
contamination	0.07	Expected proportion of anomalies in the dataset
max_features	0.5	Proportion of features randomly selected for training each tree
random_state	19	Seed for reproducibility of results

Other hyperparameters included a batch size for better memory usage, transformer layers to balance the model complexity and performance, hidden layers, and attention heads to capture different representations within the logs. Additionally, we employed an early stopping to avoid overfitting, completing training when the validation loss did not improve for some consecutive epochs. Because LogBERT is trained in a self-supervised manner exclusively on normal log windows, the objective is to model the distribution of normal behavior rather than perform supervised classification. In such anomaly detection settings, classical k-fold cross-validation is less aligned with the learning paradigm, as anomalous windows are excluded from training by design and the task does not involve discriminative class learning. Instead, we allocated a fixed proportion of normal windows for validation and applied early stopping based on validation loss. This holdout strategy ensures that the model generalizes across unseen normal patterns while preserving the one-class anomaly detection assumption. Throughout training, we continuously saved the model with the lowest validation loss for later use in log anomaly detection tasks.

4.5. Experiments

Several experiments were conducted to analyze the performance of the proposed anomaly detection system. These experiments involved parameter tuning of LogBERT to find the most optimal performance on log anomaly detection within Linux system logs, comparison with simple baseline supervised and unsupervised models, and testing its ability to generalize to unseen data. Fig. 4 shows a flowchart of the anomaly detection pipeline with colors to guide its interpretation. The green block represents the input data and marks the initial step of the pipeline. The yellow blocks represent the sequential processes involved, from cleaning and parsing the data to the core task of anomaly detection. And purple blocks indicate the outputs produced during the

Table 9
PCA parameters.

Parameter	Value	Description
n_components	10	Proportion of variance explained by selected components
c_alpha	1.7507	Threshold for reconstruction error to detect anomalies

Table 10
LSTM parameters.

Parameter	Value	Description
n_labels	2	Number of output labels (normal and abnormal)
hidden_size	100	Hidden layers
num_directions	2	Number of directions in LSTM
topk	9	The top-k predictions to consider when classifying anomalies
device	cpu	Device for model computation

Table 11
Decision-tree parameters.

Parameter	Value	Description
criterion	Entropy	Evaluates the quality of splits in the tree
max_depth	10	Maximum depth of the tree to prevent overfitting
max_features	0.3	Proportion of features considered at each split
class_weight	Balanced	Specifies if weights should be applied to classes
min_samples_split	2	Minimum samples to split a node
min_samples_leaf	10	Minimum samples at a leaf node

different stages of the pipeline, including the training and testing sets, the vocabulary, anomalous windows, and evaluation metrics to assess the model's performance.

4.5.1. Baselines

To establish a comparative baseline for this project, we implemented anomaly detections using IF, PCA, LSTM, and decision tree as traditional methods (see Sections 3.2.2 to 3.2.5). The selection of these models aligns with the existing literature [6,14,15,22] and supports a balanced comparison between unsupervised and supervised approaches. To ensure a fair performance comparison, parameter tuning for these baselines was conducted using ParameterGrid⁷ method, which systematically searches over a specific grid of parameters to identify the most optimal values.

We chose IF as an unsupervised baseline due to its ability to isolate anomalies in high-dimensional data [33]. The tuned parameters are shown in Table 8.

We chose PCA as another unsupervised baseline because of its ability to reduce the dimensionality of the data while still keeping a high variance. The tuning for PCA is shown in Table 9.

LSTM was mainly selected as a baseline model due to its ability to capture temporal dependencies in sequential data (see Table 10).

Finally, the decision tree was selected as a supervised baseline model, as it provided a straightforward solution for anomaly detection under labeled conditions. Table 11 shows the detailed parameters.

Although this research focused primarily on unsupervised models, including this simple supervised model allowed for performance comparison under labeled conditions. These tuning efforts ensured a fair and robust comparative analysis, thereby minimizing biases that may

⁷ ParameterGrid is a utility from scikit-learn that generates all combinations of parameter values from a grid. More information: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.ParameterGrid.html.

Table 12
Model configuration and training hyperparameters.

Category	Parameter	Final value
Window/sequence	window_size	128 tokens
	seq_len, max_len	2048 tokens
	Temporal interval	15 s
Sampling ratios	train:valid:test	1 : 0.1 : 1
Network	layers/hidden/heads	4/256/4
Optimization	batch_size	8
	epochs/early stop	200/10
	lr	1×10^{-3}
Masking	mask_ratio	0.50

favor our main model due to its optimized parameters. Consequently, these baselines provided relevant benchmarks for testing our model's ability to detect anomalies in the log datasets.

4.5.2. Parameter tuning

Parameter tuning followed the cyclical *Design and Development–Evaluation* and *Modeling–Evaluation* loops of DSR and CRISP-ML methodologies. Each tuning iteration represented a micro-cycle where we refined the model, demonstrated its behavior on the validation split, and evaluated its performance against test data before the next cycle. Table 12 lists the main parameters used for the model.

The model parameters were tuned based on the physical capabilities of the host machine from Company X. We used a workstation running Red Hat Enterprise Linux 9.5 and equipped with an Intel Xeon W-2123 processor and an NVIDIA Quadro P2000 GPU offering 5 GB of GDDR5 memory.

The tuning process began with the window that feeds the LLM. Although increasing self-attention would have allowed the model to have a larger window size, transformer-based architectures suffer quadratic memory usage with longer sequences, as each self-attention layer computes interactions with all tokens, and this results in a larger full attention matrix whose size increases with the window size [36]. To avoid exceeding hardware constraints, especially with the limited VRAM in the Quadro P2000 GPU, we selected a fixed window of 128 tokens for training. This size was sufficient to capture meaningful dependencies in the data while keeping the memory within manageable limits. Moreover, this setup supported a real-time scenario that required low latency in anomaly detection.

Initially, the model was configured with a maximum sequence length of 512 tokens. However, our evaluation showed that log windows exceeded this length, meaning that they were being truncated at the 512th token, which resulted in the loss of potentially relevant information. To address this, the tensor capacity was increased to 2048 tokens. This change enabled the model to receive entire log sequences without truncation during prediction. Importantly, while training still used 128-token sliding windows to control computational cost, we configured inference to allow the full window to be processed as a single sequence up to the 2048-token limit. This decoupling of the training context window from the input tensor capacity provided a balance between computational efficiency during training and complete context retention during prediction.

To further improve the quality of context in the input, we tested the model in different time intervals. Guo et al. [6] set two intervals: a 5-minute window with a 60-second overlap and a 1-minute window with a 30-second overlap. In our case, the goal was to adapt this approach for real-time detection, where faster detection with low latency was crucial. We applied sliding windows of 60, 30, and 15 s each with a fixed overlap of 10 s. Although longer windows offer richer context by capturing more logs per window, they also introduce latency. In contrast, shorter windows enable faster detection, which could potentially reduce the model's ability to accurately identify anomalies.

Similarly to what [6] did, we conducted training and validation using 100% of the available training windows, with a 10% split reserved

for validation during training. We also used a masking ratio of 50% within each window, randomly masking tokens to enable the model to learn context more efficiently following the principles of the MLM task of BERT.

The network architecture consisted of 4 Transformer layers, each with 4 attention heads and a hidden size of 256. The model was trained on 200 epochs with early stopping at 10 consecutive epochs if the validation loss had not improved. We performed optimization using the Adam optimizer⁸ with parameters $\beta_1 = 0.9$ and $\beta_2 = 0.999$, and a learning rate of 0.001. Then we used a batch size of 8 to balance throughput and memory usage on the available GPU.

4.5.3. Anomaly detection

After training, we tested our model for anomalous log window detection (see Fig. 5). The idea behind this was that since LogBERT was trained on normal sequences, it could achieve high prediction accuracy on normal log sequences while struggling with anomalous patterns, which would allow us to distinguish between normal and abnormal system behavior. This section aligns with the *Demonstration* and *Evaluation* steps of the DSR methodology in Section 3.1 and the *Evaluation* step of the CRISP-ML methodology in Section 3.1. The focus is to demonstrate the model's functionality by evaluating its performance against the defined business objectives.

The detection process began by initializing our model, where we loaded the best-performing model along with the vocabulary and the test datasets, both containing normal and abnormal log sequences. The data was chunked into windows with a maximum length of 2048 tokens and wrapped into a PyTorch dataset for batch processing. Moreover, we applied masking to 50% of the tokens within each window to evaluate the model's ability to predict original tokens.

During the scoring step, we conducted the core of our anomaly detection process, where each batch of windows experienced a forward pass through the model. For every masked token, the model generated predictions of what the original token should be, using the probability distribution calculated following Eq. (1). The model contained a candidate set with the top 10 highest probabilities and the predicted tokens, and it checked whether the predictions matched the actual masked token. If the real token was in the candidate set, then the predicted event was considered normal; otherwise, if the observed token was not in the candidate set, then the predicted event would be flagged as abnormal. To classify a sequence as anomalous, we applied a mask-error threshold of 0.1. This meant that if more than 10% of the masked tokens in a sequence were incorrectly predicted, the sequence was flagged as anomalous. We measured the effectiveness of this approach using standard evaluation metrics, including a confusion matrix, precision, recall, and F1-score.

In addition to the quantitative evaluation, we performed a qualitative assessment with domain experts. This assessment involved the inspection of flagged sequences by the model to validate its results.

Finally, all the anomalous sequences detected through the evaluation process were saved into a structured dataset for further analysis. For each flagged sequence, the corresponding anomalous EventIds were stored. To enable contextual analysis, we extracted the first 10 and last 5 messages surrounding each detected anomaly and saved them in the dataset. We did this to reduce the original sequence to get a more focused context, allowing us to isolate the most relevant portion of the sequence. However, these values (10 and 5 messages, respectively) are configurable and can be adjusted by the user according to specific analytical or operational needs. Overall, this enriched output provides a foundation for later exploratory analysis, including, for example, future root cause investigation and remediation of the anomalies.

⁸ Adam (Adaptive Moment Estimation) is an optimization algorithm that combines the benefits of momentum and adaptive learning rates. More information: <https://pytorch.org/docs/stable/generated/torch.optim.Adam.html>.

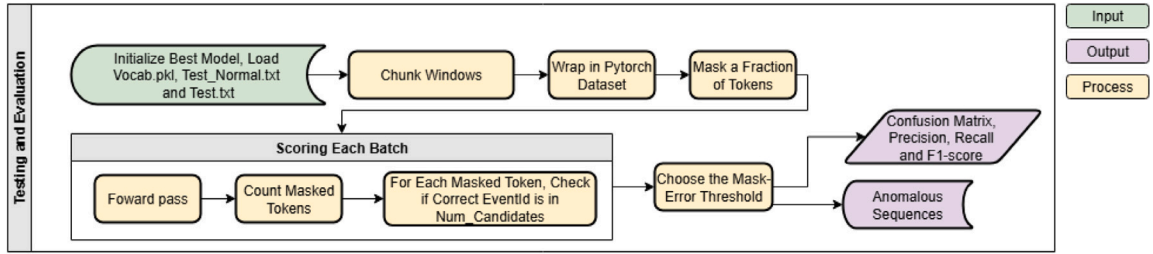


Fig. 5. Evaluation flowchart.

Table 13

Performance for minimum number of tokens in a window.

Min tokens per window	Precision (%)	Recall (%)	F ₁ -score (%)	Training time (s)
1	3.17	100	6.15	241
10	92.54	89.86	91.18	4628

5. Results

This section describes the findings of the experiments conducted to evaluate the performance of our proposed model for log anomaly detection within the AIOps framework, aiming to provide a clear understanding of the model's efficacy, particularly in real-time log anomaly detection. We structured the experiments into distinct phases, each aimed at progressively optimizing the model parameters and performance. Initially, we evaluated the impact of the minimum number of tokens in a window on model accuracy. Following that, we conducted experiments to identify the optimal time window configuration, considering varying intervals and overlaps. Finally, we optimized the mask ratio, a critical parameter in LogBERT, to improve the model predictions. We continued by analyzing the training and validation dynamics of the best-performing configuration, followed by a comparative analysis against baseline models, including IF, PCA, LSTM, and decision tree. We conclude by developing a qualitative assessment to validate the results of our model from the perspective of domain experts.

5.1. Anomaly detection results

5.1.1. Optimizing token window size

The first step in the experimental pipeline involved analyzing how the number of tokens influenced the model's performance. Initially, inspired by the approach of Guo et al. [6], who deleted log sequences with fewer than 10 tokens, potentially omitting valuable anomalies, we replicated their setup with a modification. Our buffering strategy ensured that every sequence maintained sufficient contextual information without omitting crucial data. We started with a minimum threshold of 1 token per window. This initial setting resulted in poor model performance due to a lack of context. Subsequently, we increased the minimum length to 10 tokens per sequence, as described in the literature, allowing each sequence to have adequate context for effective pattern recognition. Table 13 describes the results for this approach.

5.1.2. Optimizing sliding time window

Once we established the minimum token threshold, we explored how different time window configurations influenced the model performance. Specifically, we compared three distinct configurations: 15-second windows with 10-second overlaps, 30-second windows with 10-second overlaps, and 60-second windows with 10-second overlaps. Table 14 summarizes the precision, recall, and training time per window for these configurations. The 15-second window with a 10-second overlap achieved the highest F1-score (94.25%).

Table 14

Performance for different window sizes.

Window size	Precision (%)	Recall (%)	F ₁ -score (%)	Training time (s)
15	95.49	93.04	94.25	1237
30	96.30	91.46	93.81	2803
60	92.54	89.86	91.18	4628

Table 15

Inference latency.

Window size (s)	Min (ms)	Q1 (ms)	Median (ms)	Q3 (ms)	Max (ms)
15	3.15	4.20	5.13	8.49	6229.50
30	4.40	10.92	13.67	55.36	5470.85
60	2.97	15.15	52.14	59.98	6347.59

5.1.3. Optimizing the mask ratio

After finding an optimal time window, we focused on tuning the mask ratio, which is a critical parameter that determines the proportion of randomly masked tokens, forcing the model to learn more robust contextual information by predicting these tokens. We evaluated mask ratios ranging from 0.1 to 1.0. For each experiment, we recorded precision, recall, and F1-score to see how masking impacted the model's ability to flag anomalies. With lower mask ratios, we tested the model's capacity to detect anomalies with minimal masking, while with higher ratios, we wanted to push the model to generalize from sparse inputs. It is important to note that we used the same masking ratios for both the training and evaluation phases. Performance peaked with a mask ratio of 0.5, where the model balanced sufficient context with challenging reconstruction. Then it declined sharply at higher mask ratios as the predicting task became more difficult. Fig. 6 details these results.

5.1.4. Inference latency analysis

Once we saw that the 15-second window with a 10-second overlap delivered the best anomaly detection performance in terms of F1-score, we evaluated all three configurations to verify the real-time requirement by measuring the inference latency. We defined latency as the wall-clock time from the moment the engine requested a log sequence via de DataLoader to the moment the model returned the classification of the sequence. This interval includes batching, tensor collation, and the forward pass. Fig. 7 shows the overall latency comparison between sliding windows, and Table 15 gives detailed statistics for each window size. The data indicates that latency generally increases with window size, meaning that longer windows lead to higher inference delays. Given these observations, the 15-second window configuration has the lowest median latency (5.13 ms) and thus best meets the real-time requirement necessary for rapid anomaly detection. Nonetheless, all three configurations show acceptable latency values.

5.1.5. Training and validation loss dynamics

Fig. 8 illustrates the training and validation loss curves of our best-performing model configuration. The graph shows a consistent decrease in both training and validation losses, which means that the model

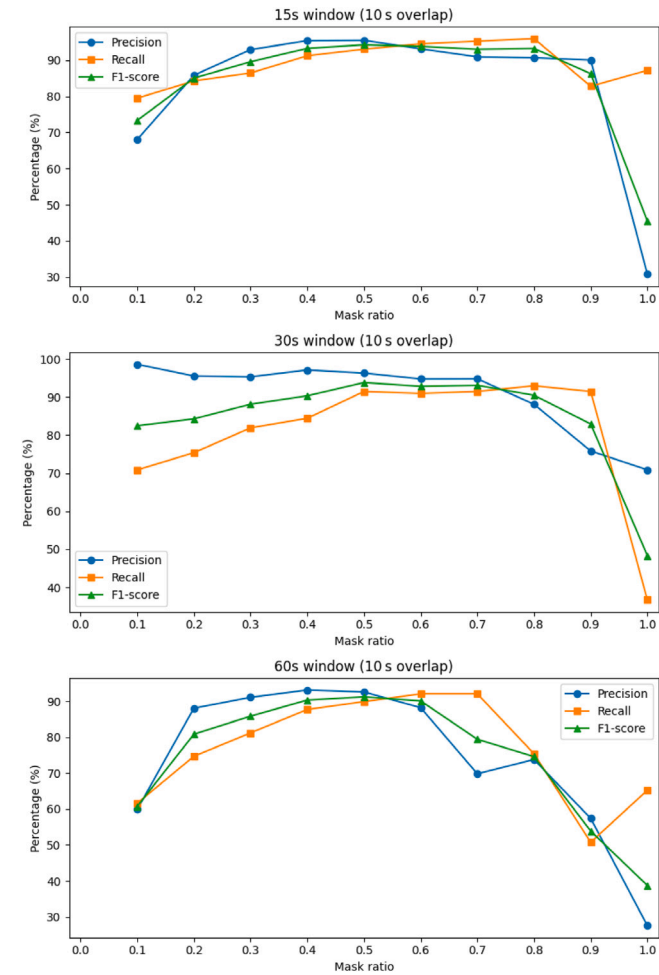


Fig. 6. Performance comparison across different mask ratios for each window configuration.

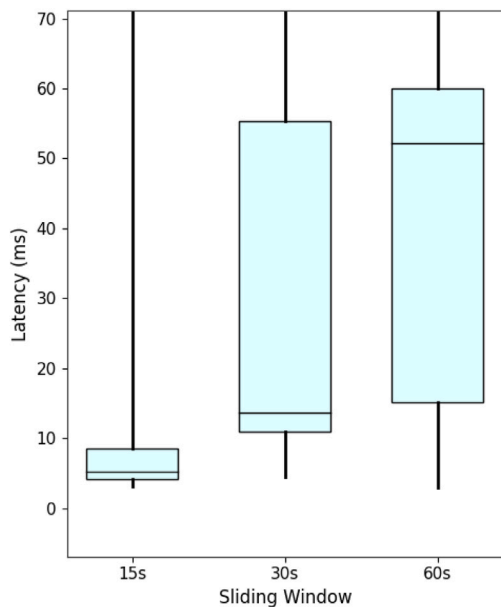


Fig. 7. Latency comparison by window size.

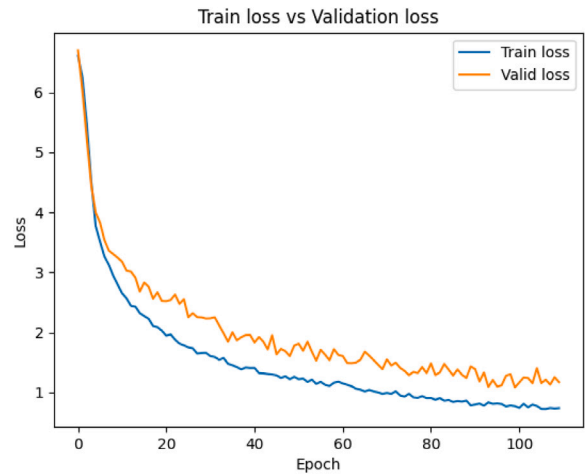


Fig. 8. Train vs. validation loss curves. The convergence of both curves suggests effective learning.

Table 16
Confusion matrix using a threshold of 0.1.

	Predicted anomalous	Predicted normal
Actual anomalous	254	19
Actual normal	12	1002

converges. Although there is a small gap between curves towards the end of training, this gap remains minimal, so we confirm that the model generalizes effectively without overfitting. The graph also shows that the training stopped in epoch 110.

5.1.6. Quantitative model comparison

To contextualize LogBERT’s performance, we compared it against four baseline models: PCA, IF, LSTM, and decision tree. This comparative analysis considered both the proprietary Company X data set and the open-source Linux dataset to evaluate the generalizability of the model in different data environments.

Since this research was a binary classification problem, normal log sequences were identified as 0, while anomalous log sequences were classified as 1. Throughout all results, the focus was primarily on the incorrectly classified log sequences.

- **True positives:** Abnormal log sequences correctly classified as anomalous.
- **True negatives:** Normal log sequences correctly classified as normal.
- **False positives:** Normal log sequences incorrectly classified as anomalous.
- **False negatives:** Abnormal log sequences incorrectly classified as normal.

Table 16 shows the confusion matrix of LogBERT’s predictions using a threshold of 0.1. The model achieved a high positive rate with minimal false negatives. In a real-world scenario, missing an actual anomaly has greater risks than incorrectly flagging a normal behavior as suspicious. If anomalies go undetected, the consequences could be severe, from security breaches to system failures that disrupt critical operations. On the other hand, a false positive may simply result in additional manual reviews or temporary alerts that can be easily fixed. Nonetheless, this does not mean that a high rate of false positives should be accepted. Therefore, despite the good balance achieved by LogBERT, there is still room for improvement to reduce unnecessary alerts while robustly identifying true anomalies.

Table 17
Experimental results on Company X, and open-source Linux datasets.

Method	%(Company X)			%(Open source)		
	Precision	Recall	F-1 score	Precision	Recall	F-1 score
PCA	98.20	60.36	37.54	33.766	96.29	50.00
Isolation Forest	94.96	68.90	79.85	83.33	18.52	30.30
LSTM	90.87	69.05	80.28	36.07	100.00	53.01
Decision Tree	96.55	51.22	66.93	92.86	96.30	94.55
LogBERT	95.49	93.04	94.25	46.58	100.00	63.55

The results in Table 17 indicate that LogBERT outperformed the baseline models when evaluated on the Company X dataset, particularly in terms of recall and precision balance, confirming its superior efficacy in anomaly detection tasks within its training domain. However, our model's performance on the open-source dataset revealed limitations in its ability to generalize to an unseen data environment. It is also important to note that we did not retrain or fine-tune the model on the open-source dataset, and only tested based on knowledge acquired from the Company X dataset. Additionally, although both datasets contained Linux logs, they originate from different systems, resulting in variations in log sequences and EventIds. These differences created new EventIds and sequences unfamiliar to the model, possibly contributing to its reduced generalization performance.

5.2. Qualitative assessment

We held a qualitative focus group session to validate the results of our model since numerical values alone could not reveal how domain experts interpret model outputs or judge their operational value. Human involvement in this experiment remained limited to an informal focus group of stakeholders. We did not collect personal or sensitive data, did not record the session, and imposed no ethical risk. The experiment, therefore, adhered to institutional guidelines for research.

We invited six stakeholders who routinely analyze log files and maintain IT platforms within Company X. During a 45-minute meeting, we presented the objectives of our research, an overview of AIOps and log anomaly detection, our model architecture, and the results collected. Then, we showed five examples of log sequences that the model had flagged as anomalous, following our definition of anomaly as any abnormal message within the internal logic of the system, mainly focusing on low-level integrity failures and mid-level service degradations. Our primary goal was to focus on how well our model classified anomalies. The group discussed each example individually, where each of the stakeholders shared their thoughts on whether they considered that sequence normal or abnormal. Then each of them independently scored each window on a 0–5 scale, where 5 denoted a definite anomaly, and 0 denoted normal behavior. Finally, we asked the stakeholders to identify potential use cases for the model.

The stakeholders assigned scores of 4–5 to four sequences, confirming the model's classification of those windows as true anomalies. They scored one sequence between 2 and 3, agreeing that it indicated a mild performance degradation rather than a failure. When we probed usability, the stakeholders proposed that several use cases could be developed from this model, not only from the performance perspective, but also from the security perspective for network traffic. They mentioned that deployment of this model would take around half a year since constant retraining is needed due to the amount of data coming into their systems. All participants were very enthusiastic about the outcomes and expressed confidence in adopting this tool, as it produced more interpretable results and reduced manual log analysis. Overall, this qualitative assessment strengthened our model validation in three different ways. First, it corroborated quantitative performance with consensus from the stakeholders, thereby increasing internal validity. Second, it highlighted a boundary case where the model signaled degradation but not a critical failure. This indicated that future work should

focus on distinguishing severity levels. Lastly, it revealed different domain applications for the model. Together, these insights bridged the gap between experimental performance and operational relevance.

6. Discussion

This section discusses the findings presented in Section 5, contextualizing them across multiple dimensions relevant to both research and practice. It begins by validating the results with prior literature to confirm the methodological consistency and reproducibility of the approach. This is followed by a detailed analysis of model performance, limitations, and latency. Beyond technical outcomes, the discussion also considers managerial implications for real-world deployment, especially in operational environments such as those of Company X. Finally, it addresses broader ethical and environmental concerns, including energy consumption, model reliability, transparency, and responsible use.

6.1. Validation through literature

Our evaluation confirmed the validity of the experimental design because it reproduced the performance that Guo et al. [6] reported on the HDFS, BGL, and Thunderbird datasets, where F1-scores reached 82.32%, 90.83%, and 96.64%, respectively. When we applied the 15-second window configuration to the Company X dataset, our model achieved an F1-score of 94.25%. A complete overview of the precision, recall, and F1-score across all datasets is reported in Table 18. Linux syslogs share a similar structure and vocabulary with Thunderbird logs, whereas BGL introduces hardware-location tags and HDFS follows a Hadoop-specific Java layout, making them less similar to our data. This structural alignment could explain why the model performance on the Company X dataset is closer to the Thunderbird benchmark than to the HDFS or BGL. The frequency of logs throughout the dataset without prolonged inactive periods and the stable distribution further supported the model objectives, as it was able to learn regular patterns and detect anomalies more effectively. Our results fall within the upper range of the original benchmarks, thus confirming that we accurately reproduced the methodology.

Additionally, Guo et al. [6] demonstrated that moderate mask ratios preserved semantic context while still being challenging for the model to reconstruct the missing information. To the best of our knowledge, no other studies beyond this work have explored the effect of different mask ratios on the model's performance. Our experiments also reached a similar peak in performance, specifically at 0.5, confirming that our implementation followed the intended learning dynamics. This outcome shows the robustness of masked log prediction across different types of datasets and suggests that the model benefits from a balance between catching enough information and task difficulty. Consequently, the results shown in Section 5 validated the methodology we followed and indicated that our approach was reproduced in a new operational context.

This analysis confirmed the methodological consistency of our approach. To further contextualize the results, we positioned the obtained performance relative to other established approaches in log anomaly detection. Classical unsupervised techniques, including PCA and Isolation Forest, obtained F1-scores of 0.741 and 0.718, respectively, on the HDFS dataset [6]. DeepLog [40] achieved an F1-score of approximately 0.93 on HDFS and 0.90 on BGL datasets. More recently, LogLLM [18] evaluated on HDFS and BGL datasets achieved F1-scores of 0.997 and 0.916, respectively. Within this context, the F1-score of 94.25% obtained on Linux syslogs demonstrated performance comparable to deep and transformer-based models reported in prior studies. Structural differences between datasets and log distributions limited direct numerical comparison. Nonetheless, the results indicated that the adaptation of LogBERT remained competitive while addressing practical operational constraints.

Table 18
LogBERT results across datasets.

	HDFS			BGL			Thunderbird			Linux		
	Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.	F1
LogBERT	87.02	78.10	82.32	89.40	92.32	90.83	96.75	96.52	96.64	95.49	93.04	94.25

6.2. Assessing log anomaly detection results

The results demonstrated that the configuration using a 15-second sliding window with a 10-second overlap achieved the best overall performance, balancing anomaly detection and inference latency. As shown in Table 14, this setup reached a F1-score of 94.25%, with a precision of 95.49% and a recall of 93.04%. In contrast, longer window configurations, 30 and 60 s, resulted in slightly lower F1-scores of 93.81% and 91.18% respectively. While these outcomes reflected a small drop in performance as the sliding window increased, it is important to note that all three approaches delivered strong results, consistently maintaining F1-scores above 90%. This showed that the model generalized well across different sliding windows. Shorter windows tended to preserve more uniform log sequences, while longer windows were more likely to include more unrelated operational events because the timestamp was longer, leading to a higher diversity of logs within the same sequence. This diversity may have made masked log prediction less reliable, as LogBERT's prediction mechanism is based primarily on sequence structure rather than the semantic context of individual logs. Moreover, the 15-second window configuration also required the shortest training time compared to the 30 and 60-second windows. This can be attributed to the shorter sequence length per window, which reduced the computational complexity per training step. As explained by Vaswani et al. [36], transformer-based models exhibit quadratic time complexity with respect to the sequence length because of the self-attention mechanism, where each token attends to every other token in the sequence. For this reason, shorter input sequences resulted in lower memory for training and also required fewer gradients for propagation.

Latency measurements also supported the 15-second window configuration as the most efficient choice. As shown in Table 15, it achieved a median inference time of 5.13 ms, considerably lower than the 13.667 ms recorded for 30-second windows and 52.14 ms for 60-second windows. These inference times included the stages of the prediction process, so from the moment a log sequence was retrieved from the DataLoader until the model returned the classification output for the sequence. We focused on inference latency to assess the model's computational efficiency independently from preprocessing. Since we handled preprocessing and inference in separate scripts, measuring combined latency would have introduced artificial delays. For this reason, we preferred to isolate inference latency as a standalone metric.

Regarding model generalizability, LogBERT showed limitations when tested on the open-source dataset. This difference in performance, with an F1-score of 63.55% on the open-source dataset compared to 94.25% on the Company X dataset, likely resulted from variations in log sequences and naming of the logs between the two Linux systems. It is important to note that we did not retrain or fine-tune the model on the open-source dataset, which limited its familiarity with new identifiers and sequence structures not present in the original training environment. In addition, this dataset spanned approximately 260 days, but it contained a limited number of logs, resulting in multiple empty timestamps. Consequently, the sliding window approach, while strictly applied, may not have accurately represented real-time conditions, as windows could remain buffered for extended periods due to these gaps, sometimes spanning days instead of seconds. Although retraining with this dataset could have improved performance, such an approach was not pursued due to the dataset's inherent limitations, which made it inadequate for robust experimentation or modification of the sliding windows. Subsequently, we conducted further testing in a tentative to

also validate our results with another sample of data from Company X gathered from one day in June. This new dataset originated from the same system, but the software had been reinstalled a few weeks earlier, according to stakeholders, which generated new log messages and introduced small changes in format and content. There were new log sequences, and the dataset contained 412 new logs that the model had not been trained on. To ensure more consistency between the old and new data, we introduced more modifications in the parsing pipeline. We stored the learned template structure of Drain [19] in a persistent file to allow continuity across parsing sessions. Additionally, we used a JSON file to maintain a consistent mapping of parsed log templates to unique identifiers. These steps allowed for minimizing discrepancies caused by reinstallation and ensured that newly parsed logs could be matched with previously observed logs whenever possible. Without retraining or fine-tuning the model, performance dropped significantly, yielding a precision of 51.22%, a recall of 87.50%, and an F1-score of 64.62%. However, after retraining the model using both the original and new datasets from Company X, performance improved substantially again, achieving a precision of 86.86%, a recall of 95.60%, and an F1-score of 91.02%. This underscored that performance degradation on external or new datasets primarily arises from unfamiliarity with specific logs and sequence structures, highlighting the necessity for retraining to ensure effective generalization.

6.3. Managerial implications

From a managerial perspective, deployment of this model presents strategic and operational advantages, particularly within the military domain where system functionality and reliability are critical during important operations. At Company X, operators must ensure all systems are fully functional before critical decisions can be made. From discussions with stakeholders, typically, IT operators and maintainers face challenges in resolving system failures, often with strict resolution windows before the system automatically reinstalls. While reinstalling might temporarily resolve the issues, it disrupts operational continuity and could result in operational downtime, compromising mission-critical decisions. By identifying and classifying anomalies, our model enables proactive rather than reactive responses, decreasing operational risks associated with system downtime and enhancing overall system reliability. It would allow maintainers to rapidly pinpoint and solve issues within the allocated time frame. Additionally, integrating LogBERT within the military IT infrastructure improves productivity among the IT personnel. By automating the log parsing and anomaly detection process, it reduces the manual workload traditionally associated with analyzing extensive volumes of unstructured log data and also boosts personnel effectiveness by eliminating tedious and repetitive tasks. We ensured reproducibility by adopting an unsupervised training pipeline consistent with prior work [6].

At the same time, practical deployment at scale introduces implementation challenges that can affect generalizability across heterogeneous systems. First, computational load may increase substantially as log volume grows, because inference must be performed continuously over sliding windows and across multiple hosts, which can create resource contention on shared infrastructure. Second, real-time performance can degrade under bursty logging behavior (e.g., during cascading failures), when the system must process a sudden spike in events without exceeding latency budgets. Third, generalizability is sensitive to system variation: changes in log templates, software versions, localization settings, or new services can induce distribution shifts that

reduce detection reliability unless retraining or incremental adaptation is performed. To mitigate these risks, an operational deployment should include capacity planning (CPU/GPU sizing and memory budgets), a streaming architecture that supports horizontal scaling, and monitoring for drift and throughput bottlenecks with triggers for re-tokenisation, periodic retraining, or targeted fine-tuning on new log formats. In addition, a tiered design can route low-risk hosts to lightweight models while reserving the full LogBERT pipeline for mission-critical components, thereby preserving real-time responsiveness without sacrificing coverage.

Regarding adaptability to changing environments, the framework can be extended with lightweight adaptation mechanisms that do not require full re-engineering. In practice, template mining and periodic vocabulary updates can keep tokenisation aligned with evolving log formats, while incremental retraining on newly observed “normal” windows can refresh the baseline without exposing sensitive data outside the organization. When drift is detected (e.g., sustained changes in reconstruction loss or masking-error rates), the system can trigger a controlled re-calibration cycle that updates thresholds and refreshes the model using recent logs, preserving operational continuity while maintaining detection fidelity.

6.4. Ethical and environmental concerns

The integration of AI into operational systems has multiple benefits, but also introduces new ethical and environmental challenges. These concerns must be carefully considered to ensure responsible development and deployment of AI-driven solutions. While these approaches can perform extremely well, it is also wise to reflect on the environmental impact of artifacts that use LLMs. Currently, one of the biggest bottlenecks in AI development is energy constraints.

For our research, we estimated that we utilized around 70 h for model development. The workstation ran Red Hat Enterprise Linux 9.5 equipped with an Intel Xeon W-2123 processor with 120 W of thermal design power, and an NVIDIA Quadro P2000 GPU with a board power of 75 W. Allowing 50 W for memory, storing, and cooling, we estimated that the training load averaged around 245 W. The resulting energy consumption equals

$$E = 0.245 \text{ kW} \times 70 \text{ h} = 17.15 \text{ kWh} \quad (3)$$

The EI is the professional membership body for the world energy [41]. The EI Statistical Review of World Energy analyzes data on world energy markets from the prior year, and it has been providing objective and comprehensive data to the energy community since 1952 [41]. According to this review, the mean grid carbon intensity in the Netherlands was $253 \text{ gCO}_2 \text{ kWh}^{-1}$. The mean carbon intensity of a country refers to the average quantity of carbon emissions produced per unit of economic output [41]. Therefore, the estimated carbon footprint of this energy use equals

$$\begin{aligned} E_{\text{CO}_2} &= E \times \text{CI} = 17.15 \text{ kWh} \times 253 \text{ gCO}_2 \text{ kWh}^{-1} = 4338.95 \text{ gCO}_2 \\ &\approx 4.34 \text{ kgCO}_2 \end{aligned} \quad (4)$$

Our project represented a modest carbon footprint if we compare it to other large-scale AI products that run daily. Nonetheless, it is important to consider that energy demand will grow in the case that this model is deployed into a continuous monitoring pipeline because the frequency of model usage will increase. Moreover, none of the papers reviewed in the existing literature explicitly calculated the carbon emissions associated with their models. Therefore, highlighting this analytical perspective strengthens the scientific contributions of our research, promoting greater awareness and transparency in the environmental impact of our solution.

In addition to environmental concerns, ML and LLM-based approaches for log anomaly detection raise a number of ethical concerns,

particularly regarding explainability, human oversight, and model reliability. One of the main limitations of LLMs is their lack of explainability, which can lead to risks of unvalidated decisions. In operational contexts, explainability is essential for maintaining trust and accountability [42,43]. We advise that IT operators and maintainers remain in the pipeline loop. The model predictions should not be treated as deterministic or standalone judgements but rather as a support that would require expert validation before making a decision. Stakeholders should discuss the most appropriate stages in the monitoring pipeline where expert validation can be integrated.

A further ethical concern we discussed with stakeholders is the risk of model misapplication. False positives may generate unnecessary alerts that increase workload, while false negatives could lead to missed or delayed responses to critical anomalies. Zerilli et al. [43] show that false alarms and misses affect trust in different ways depending on their perceived consequences. In our case, this risk grows if operators assume that the output will always be correct. Therefore, to address this risk, the prediction threshold of the model should be tuned and evaluated under realistic operational conditions that reflect the complexity and diversity of the log data within Company X.

Transparency is also crucial. It refers to the need to explain, interpret, and reproduce the decisions of an AI system to different stakeholders [44]. Users must be informed about the nature of the trained data used for the development of the model, its limitations, and its expected behavior under conditions where the input has not been seen [44]. This limitation was raised during our focus group discussion, where stakeholders emphasized that logs are generated from a heterogeneous and evolving set of systems. The model may not generalize to new logs or sequences as we discussed in Section 6.2, which introduces risks if this limitation is not communicated.

Lastly, although it lies outside the direct scope of our research, the possibility of unethical use must also be acknowledged. If the model were to be deployed to sensitive infrastructure without safeguards and proper monitoring, it could lead to damage to the public image of Company X, including operational and even legal consequences. The development and deployment of this system should be guided not only by performance metrics but also by principles of accountability, transparency, and responsible use.

7. Conclusion

This study investigated the use of large language models (LLMs) for log anomaly detection within the AIOps domain, with a particular focus on the military context where data confidentiality and operational resilience are paramount. The research was guided by the aim to design and implement an LLM-based approach for log anomaly detection in AIOps pipelines within the military domain, ensuring data confidentiality and adapting to new Linux syslog scenarios.

The contributions of this work are threefold. First, we designed and implemented an anomaly detection framework built on LogBERT, a self-supervised model trained exclusively on normal logs. This architecture was selected to balance scalability, efficiency, and the real-time detection requirements outlined by our stakeholders. The framework effectively addressed data confidentiality concerns by incorporating strict preprocessing to remove sensitive information and by deploying the model locally, thereby avoiding external dependencies and mitigating exposure risks.

Second, the methodology combined iterative experimentation with validation against baseline models and established benchmarks. The experimental results demonstrated that a 15-second sliding window configuration with a 10-second overlap provided the best trade-off between detection accuracy and inference latency. This configuration is especially relevant for real-time operational environments. While LogBERT outperformed traditional anomaly detection methods, it showed limited generalizability to new datasets without retraining, highlighting an area where adaptation mechanisms are needed.

Third, the study complemented quantitative evaluation with qualitative validation by domain experts. Stakeholders confirmed the operational benefits of the approach, such as improved system reliability, reduced manual monitoring effort, and potential extensions beyond system performance monitoring to applications in network security. Ethical and environmental considerations were also incorporated by estimating the carbon footprint of training and emphasizing the importance of explainability, transparency, and responsible deployment.

7.1. Limitations and future research

Several limitations should be acknowledged. The quality of the datasets posed challenges, as they were unstructured and unlabeled. In practice, this meant that log templates, message delimiters, and severity fields were not consistently formatted across sources, requiring additional normalization and increasing the risk that semantically equivalent events were represented as distinct tokens. The reliance on manually defined ground truth introduced potential inaccuracies and bias in performance evaluation, despite validation with experts. More specifically, labels were derived from heuristic rules and expert judgement on a limited set of time windows, which may under-represent rare but operationally critical anomalies and can favor patterns that are more easily observable in the available logs. Computational constraints further limited the exploration of larger models and extensive hyperparameter optimization, leading to the adoption of a lightweight architecture. This constrained the number of training runs, the breadth of window/overlap configurations tested, and the extent to which we could compare alternative transformer sizes or ablation variants under identical conditions. Moreover, the model demonstrated limited generalizability across system variations, requiring retraining when log formats changed. For instance, changes in syslog headers, timestamp conventions, or the introduction of new services altered token distributions and sequence structure, which reduced performance unless the tokenizer/vocabulary and model were updated with additional normal-log training data. Finally, confidentiality restrictions prevented access to more detailed log data, constraining the depth of explanatory analysis. In particular, the need to redact host identifiers, user references, and mission-relevant context limited the ability to link anomalies to specific assets or operational workflows, and reduced the feasibility of richer root-cause annotations beyond message-level patterns.

These limitations open promising directions for future research. One avenue is the extension of anomaly detection with post-detection exploratory analysis. Integrating retrieval-augmented generation (RAG) with an LLM could link detected anomalies to historical reports and solutions, enriching interpretability and supporting root cause analysis. In addition, future work could evaluate such a retrieval layer under realistic operational constraints, including delayed incident labels, incomplete runbooks, and evolving threat tactics, to determine when retrieved evidence is sufficiently reliable for decision support. A complementary step is to establish evaluation protocols for root cause analysis quality, combining automated metrics (e.g., evidence precision/recall and citation faithfulness) with structured expert rubrics that reflect operational impact. Another direction lies in refining the sliding-window strategy by incorporating adaptive techniques sensitive to log volume and time intervals. For example, dynamic windowing could be triggered by bursts in log rate, changes in service topology, or the detection of precursor events, thereby reducing both alert fatigue and missed anomalies in high-load periods. Further, feature engineering, such as incorporating delta time or sequence length, could improve predictive performance by capturing more complex system behaviors. More broadly, domain-informed features that reflect deployment realities (e.g., service identifiers, process lineage, and dependency context) could support stronger cross-host generalization without exposing sensitive content. Future research could also investigate privacy-preserving

or federated training across enclaves to improve robustness while maintaining confidentiality, alongside lightweight continual-learning strategies that mitigate catastrophic forgetting when log formats evolve. Finally, closer integration of expert domain validation throughout the anomaly analysis process would strengthen interpretability and ensure alignment with operational objectives. This could take the form of human-in-the-loop feedback loops where analysts confirm or reject anomaly clusters, enabling active learning and improving calibration of alert thresholds over time, while producing auditable artefacts that support governance and accountability requirements.

CRedit authorship contribution statement

Miguel De la Cruz Cabello: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Tiago Prince Sales:** Writing – review & editing, Supervision, Resources, Formal analysis. **Marcos R. Machado:** Writing – review & editing, Validation, Supervision, Project administration, Investigation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

References

- [1] B.A.P. Prasad, C. Rich, Licensed for distribution market guide for AIOps platforms, 2019, URL: <https://www.gartner.com/doc/reprints?id=1-5VVK3KA&ct=181130&st=sb>.
- [2] K.M. Sallam, I. Alrashdi, A.M. Ali, I. Radwan, M. Abdel-Basset, A robust framework utilizing artificial intelligence to enhance services in smart city environments, *Clust. Comput.* 28 (2025) 741, <http://dx.doi.org/10.1007/s10586-025-05431-9>.
- [3] I.J. Mansour, M.B.M. Rejab, H.B. Mahdin, Review in adoption of DevOps, AIOps, DataOps, GitOps, MLOps in IT MLEs in Germany, *Int. J. Eng. Trends Technol.* 72 (2024) 64–76, <http://dx.doi.org/10.14445/22315381/IJETT-V72I12P106>.
- [4] T. Zhang, Z. Jiang, S. Bai, T. Zhang, L. Lin, Y. Liu, J. Ren, RAG4ITOps: A supervised fine-tunable and comprehensive RAG framework for IT operations and maintenance, 2024, URL: <http://arxiv.org/abs/2410.15805>.
- [5] W. Ding, M. Abdel-Basset, A.M. Ali, N. Moustafa, Large language models for cyber resilience: A comprehensive review, challenges, and future perspectives, *Appl. Soft Comput.* 170 (2025) 112663, <http://dx.doi.org/10.1016/j.asoc.2024.112663>.
- [6] H. Guo, S. Yuan, X. Wu, LogBERT: Log anomaly detection via BERT, 2021, URL: <http://arxiv.org/abs/2103.04475>.
- [7] Y. Dang, Q. Lin, P. Huang, AIOps: Real-world challenges and research innovations, in: *Proceedings - 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion, ICSE-COMpanion 2019*, Institute of Electrical and Electronics Engineers Inc., 2019, pp. 4–5, <http://dx.doi.org/10.1109/ICSE-Companion.2019.00023>.
- [8] L. Zhang, T. Jia, M. Jia, Y. Wu, A. Liu, Y. Yang, Z. Wu, X. Hu, P.S. Yu, Y. Li, A survey of AIOps for failure management in the era of large language models, 2024, URL: <http://arxiv.org/abs/2406.11213>.
- [9] L. Poenaru-Olaru, N. Karpova, L. Cruz, J.S. Rellermeier, A.V. Deursen, Is your anomaly detector ready for change? Adapting aiops solutions to the real world, in: *Proceedings - 2024 IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI, CAIN 2024*, Association for Computing Machinery, Inc, 2024, pp. 222–233, <http://dx.doi.org/10.1145/3644815.3644961>.
- [10] P. Notaro, J. Cardoso, M. Gerndt, A survey of AIOps methods for failure management, *ACM Trans. Intell. Syst. Technol.* 12 (2021) <http://dx.doi.org/10.1145/3483424>.
- [11] Y. Duan, H. Bao, G. Bai, Y. Wei, K. Xue, Z. You, Y. Zhang, B. Liu, J. Chen, S. Wang, Z. Ou, Learning to diagnose: Meta-learning for efficient adaptation in few-shot AIOps scenarios, *Electron. (Switzerland)* 13 (2024) <http://dx.doi.org/10.3390/electronics13112102>.

- [12] J. Diaz-De-Arcaya, A.I. Torre-Bastida, G. Zárate, R. Miñón, A. Almeida, A joint study of the challenges, opportunities, and roadmap of MLOps and AIOps: A systematic survey, *ACM Comput. Surv.* 56 (2023) <http://dx.doi.org/10.1145/3625289>.
- [13] A. Vitui, T.-H. Chen, Empowering AIOps: Leveraging large language models for IT operations management, 2025, URL: <http://arxiv.org/abs/2501.12461>.
- [14] H. Guo, J. Yang, J. Liu, J. Bai, B. Wang, Z. Li, T. Zheng, B. Zhang, J. Peng, Q. Tian, LogFormer: A pre-train and tuning pipeline for log anomaly detection, 2024, URL: www.aiai.org.
- [15] F. Hadadi, Q. Xu, D. Bianculli, L. Briand, Anomaly detection on unstable logs with GPT models, 2024, URL: <http://arxiv.org/abs/2406.07467>.
- [16] M. Mehrabi, A. Hamou-Lhadi, H. Moosavi, The effectiveness of compact fine-tuned LLMs in log parsing, 2024, URL: <https://huggingface.co/spaces/HuggingFaceH4/open>.
- [17] J. Su, C. Jiang, X. Jin, Y. Qiao, T. Xiao, H. Ma, R. Wei, Z. Jing, J. Xu, J. Lin, Large language models for forecasting and anomaly detection: A systematic literature review, 2024, URL: <http://arxiv.org/abs/2402.10350>.
- [18] W. Guan, J. Cao, S. Qian, J. Gao, C. Ouyang, LogLLM: Log-based anomaly detection using large language models, 2024, URL: <http://arxiv.org/abs/2411.08561>.
- [19] P. He, J. Zhu, Z. Zheng, M.R. Lyu, Drain: An online log parsing approach with fixed depth tree, in: *Proceedings - 2017 IEEE 24th International Conference on Web Services, ICWS 2017*, Institute of Electrical and Electronics Engineers Inc., 2017, pp. 33–40, <http://dx.doi.org/10.1109/ICWS.2017.13>.
- [20] H. Guo, J. Yang, J. Liu, L. Yang, L. Chai, J. Bai, J. Peng, X. Hu, C. Chen, D. Zhang, X. Shi, T. Zheng, L. Zheng, B. Zhang, K. Xu, Z. Li, OWL: A large language model for IT operations, 2023, URL: <http://arxiv.org/abs/2309.09298>.
- [21] R. Xu, K. Ding, Large language models for anomaly and out-of-distribution detection: A survey, 2024, URL: <http://arxiv.org/abs/2409.01980>.
- [22] C. Almodovar, F. Sabrina, S. Karimi, S. Azad, LogFIT: Log anomaly detection using fine-tuned language models, *IEEE Trans. Netw. Serv. Manag.* 21 (2024) 1715–1723, <http://dx.doi.org/10.1109/TNSM.2024.3358730>.
- [23] Y. Liu, S. Tao, W. Meng, J. Wang, W. Ma, Y. Zhao, Y. Chen, H. Yang, Y. Jiang, X. Chen, Interpretable online log analysis using large language models with prompt strategies, 2023, URL: <http://arxiv.org/abs/2308.07610>.
- [24] J. Wang, G. Chu, J. Wang, H. Sun, Q. Qi, Y. Wang, J. Qi, J. Liao, LogExpert: Log-based recommended resolutions generation using large language model, in: *Proceedings - International Conference on Software Engineering, IEEE Computer Society*, 2024, pp. 42–46, <http://dx.doi.org/10.1145/3639476.3639773>.
- [25] G. No, Y. Lee, H. Kang, P. Kang, Training-free retrieval-based log anomaly detection with pre-trained language model considering token-level information, *Eng. Appl. Artif. Intell.* 133 (2024) <http://dx.doi.org/10.1016/j.engappai.2024.108613>.
- [26] O. Ovadia, M. Brief, M. Mishaali, O. Elisha, Fine-tuning or retrieval? Comparing knowledge injection in LLMs, 2023, URL: <http://arxiv.org/abs/2312.05934>.
- [27] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, H. Wang, Retrieval-augmented generation for large language models: A survey, 2023, URL: <http://arxiv.org/abs/2312.10997>.
- [28] A. Vizniuk, G. Diachenko, I. Laktionov, A. Siwocha, M. Xiao, J. Smolag, A comprehensive survey of retrieval-augmented large language models for decision making in agriculture: Unsolved problems and research opportunities, *J. Artif. Intell. Soft Comput. Res.* 15 (2024) 115–146, <http://dx.doi.org/10.2478/jaiscr-2025-0007>.
- [29] J. Jeon, Y. Sim, H. Lee, C. Han, D. Yun, E. Kim, S.L. Nagendra, M.B. Jun, Y. Kim, S.W. Lee, J. Lee, ChatCNC: Conversational machine monitoring via large language model and real-time data retrieval augmented generation, *J. Manuf. Syst.* 79 (2025) 504–514, <http://dx.doi.org/10.1016/j.jmsy.2025.01.018>.
- [30] S. Chen, H. Li, Z. Zhang, Z. Sun, Y. Cheng, OMAI: A specialized large language model for operational maintenance, 2024, URL: <https://pos.sissa.it/>.
- [31] J. Vom Brocke, A. Hevner, A. Maedche, *Introduction to design science research, in: Design Science Research. Cases*, Springer, 2020, pp. 1–13.
- [32] S. Studer, T.B. Bui, C. Drescher, A. Hanuschkin, L. Winkler, S. Peters, K.-R. Mueller, Towards CRISP-ML(Q): A machine learning process model with quality assurance methodology, 2020, URL: <http://arxiv.org/abs/2003.05155>.
- [33] F.T. Liu, K.M. Ting, Z.H. Zhou, Isolation forest, in: *Proceedings - IEEE International Conference on Data Mining, ICDM*, 2008, pp. 413–422, <http://dx.doi.org/10.1109/ICDM.2008.17>.
- [34] Z.-H. Zhou, *Machine Learning*, Springer Nature, 2021.
- [35] R.C. Staudemeyer, E.R. Morris, Understanding LSTM – a tutorial into long short-term memory recurrent neural networks, 2019, URL: <http://arxiv.org/abs/1909.09586>.
- [36] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, 2017, URL: <http://arxiv.org/abs/1706.03762>.
- [37] J. Devlin, M.-W. Chang, K. Lee, K.T. Google, A. Language, BERT: Pre-training of deep bidirectional transformers for language understanding, 2019, URL: <https://arxiv.org/pdf/1810.04805>.
- [38] J. Zhu, S. He, P. He, J. Liu, M.R. Lyu, Loghub: A large collection of system log datasets for AI-driven log analytics, 2023, URL: <http://arxiv.org/abs/2008.06448>.
- [39] P. He, J. Zhu, Z. He, M.R. Lyu, LogHub: A large collection of system log datasets, 2020, <https://github.com/logpai/loghub/tree/master/Linux>.
- [40] Min Du, Feifei Li, Guineng Zheng, Vivek Srikumar, DeepLog: Anomaly detection and diagnosis from system logs through deep learning, in: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS, ACM*, 2017, pp. 1285–1298, <http://dx.doi.org/10.1145/3133956.3134015>.
- [41] Statistical review of world energy, 2024, URL: https://www.energyinst.org/_data/assets/pdf_file/0006/1542714/684_EI_Stat_Review_V16_DIGITAL.pdf.
- [42] K. Porayska-Pomsta, W. Holmes, S. Nemorin, *The ethics of AI in education handbook of artificial intelligence in education*, 2021.
- [43] J. Zerilli, U. Bhatt, A. Weller, How transparency modulates trust in artificial intelligence, 2022, <http://dx.doi.org/10.1016/j.patter.2022.100455>.
- [44] D. Kaur, S. Uslu, K.J. Rittichier, A. Durrezi, Trustworthy artificial intelligence: A review, 2023, <http://dx.doi.org/10.1145/3491209>.